Roadmap

Learning Path

Fundamentals → Core Building Blocks → Distributed Systems → Design Patterns → Technology Deep Dives → System Designs

---

1. System Design Fundamentals

- Functional Requirements
- Non-Functional Requirements
- Back-of-the-Envelope Estimation
- Latency vs Throughput
- Availability
- Scalability
- Reliability
- Consistency
- Stateless vs Stateful Systems
- Horizontal vs Vertical Scaling
- Synchronous vs Asynchronous Communication
- Push vs Pull

Tip: Always start a system design by understanding what the system needs before deciding what technologies to use.

---

2. Networking Fundamentals

Only learn networking concepts directly useful for system design.

- DNS
- TCP
- HTTP/HTTPS
- TLS Handshake
- HTTP/1.1
- HTTP/2
- HTTP/3
- WebSockets
- Long Polling
- Reverse Proxy
- CDN

Tip: Understand the request journey from client to server rather than going deep into networking theory.

---

3. Databases & Storage

Database Fundamentals

- SQL vs NoSQL
- Data Modeling
- Normalization vs Denormalization
- Indexing
- Transactions
- ACID
- Isolation Levels
- Locks
- MVCC

Relational Databases

- PostgreSQL / MySQL
- Read Replicas
- Replication
- Partitioning
- Sharding

NoSQL

- Key-Value Databases
- Document Databases
- Wide-Column Databases

Representative technologies:

- DynamoDB
- MongoDB
- Cassandra

Distributed Data

- Leader-Follower Replication
- Synchronous vs Asynchronous Replication
- Replication Lag

- Sharding
- Consistent Hashing
- Hot Partitions
- Rebalancing

Data Consistency

- Strong Consistency
- Eventual Consistency
- Quorum
- Read-After-Write Consistency
- Read Replicas and Stale Reads

Tip: Learn database concepts first, then understand how individual databases implement those concepts.

---

4. Caching

Fundamentals

- Why Caching
- Cache Hit
- Cache Miss
- Cache Hit Ratio

Cache Strategies

- Cache-Aside
- Read-Through
- Write-Through
- Write-Behind

Cache Management

- TTL
- Cache Invalidation
- Eviction Policies
- Cache Warming

Cache Problems

- Cache Stampede
- Cache Penetration
- Cache Avalanche
- Hot Keys
- Cache Consistency
- Distributed Cache

Redis

- Redis Data Structures
- Redis as Cache
- Redis Replication
- Redis Cluster
- Redis Persistence
- Redis Pub/Sub
- Redis Streams
- Distributed Locks

Tip: Every caching strategy involves a tradeoff between performance, consistency, and complexity.

---

5. Load Balancing & Traffic Management

Load Balancing

- Load Balancer
- L4 vs L7 Load Balancing
- Load Balancing Algorithms
- Health Checks
- Sticky Sessions

API Gateway & Reverse Proxy

- API Gateway
- Reverse Proxy
- Request Routing
- Authentication at Gateway
- Rate Limiting at Gateway

Rate Limiting

- Fixed Window

- Sliding Window
- Token Bucket
- Leaky Bucket
- Distributed Rate Limiting

CDN

- CDN
- Edge Locations
- Edge Caching
- CDN Invalidation
- Static Content Delivery

Tip: Always know where each component sits in the request flow.

Client → DNS → CDN → Load Balancer → API Gateway / Reverse Proxy → Services

---

6. Messaging & Asynchronous Processing

Fundamentals

- Why Asynchronous Processing
- Message Queues
- Event Streaming
- Push vs Pull
- Producers
- Consumers
- Brokers

Message Queues

- RabbitMQ
- Amazon SQS
- Message Acknowledgements
- Message Ordering
- Visibility Timeout
- Retries
- Dead Letter Queues

Kafka

- Kafka Architecture
- Topics
- Partitions
- Producers
- Consumers
- Consumer Groups
- Offsets
- Consumer Rebalancing
- Replication
- Ordering
- Retention
- Replay
- Log Compaction

Delivery Semantics

- At-Most-Once
- At-Least-Once
- Exactly-Once
- Duplicate Messages
- Idempotent Consumers

Async Problems

- Backpressure
- Slow Consumers
- Poison Messages
- Retry Storms
- Ordering Problems

Tip: When using asynchronous systems, always think about duplicates, ordering, retries, failures, and backpressure.

---

7. Distributed Systems Fundamentals

- Why Distributed Systems Are Hard
- Partial Failures
- Network Failures
- Network Partitions
- CAP Theorem
- PACELC
- Consistency vs Availability
- Replication

- Quorum
- Split Brain
- Leader Election
- Distributed Locks
- Clock Problems
- Idempotency

Tip: Do not memorize CAP. Learn what tradeoff you are actually making when a network partition happens.

---

8. Reliability & Fault Tolerance

Timeouts & Retries

- Timeouts
- Retries
- Exponential Backoff
- Jitter
- Retry Storms

Failure Isolation

- Circuit Breaker
- Bulkhead
- Cascading Failures

Graceful Failure

- Graceful Degradation
- Fallback
- Load Shedding
- Fail Fast

Recovery

- Failover
- Dead Letter Queue
- Disaster Recovery Basics

Tip: For every external dependency, ask: "What happens if this becomes slow or completely unavailable?"

9. Service Architecture

Architecture Styles

- Monolith
- Modular Monolith
- Microservices

Microservice Design

- Service Boundaries
- Database Per Service
- Service Discovery
- Service Ownership

Communication

- Synchronous Communication
- Asynchronous Communication
- REST
- gRPC
- Event-Based Communication

APIs

- API Design
- API Versioning
- Backward Compatibility
- Pagination
- Error Handling

Architecture Components

- Backend for Frontend
- API Gateway

Tip: Microservices are not automatically better. Learn when a monolith is the better choice.

10. Core Design Patterns

Learn these after understanding the underlying fundamentals.

Data Consistency Patterns

- Saga Pattern
- Choreography vs Orchestration
- Compensating Transactions
- Outbox Pattern
- Transactional Outbox
- Change Data Capture
- Idempotency Pattern
- Eventual Consistency
- Two-Phase Commit

Event-Driven Patterns

- Event-Driven Architecture
- Publish-Subscribe
- CQRS
- Event Sourcing
- Webhooks

Resilience Patterns

- Circuit Breaker
- Retry Pattern
- Bulkhead Pattern
- Graceful Degradation
- Dead Letter Queue

Scalability Patterns

- Sharding
- Partitioning
- Consistent Hashing
- Fan-Out on Write
- Fan-Out on Read
- Fan-In
- Batching
- Backpressure

Architecture Patterns

- API Gateway Pattern
- Backend for Frontend
- Aggregator Pattern
- Sidecar Pattern
- Strangler Fig Pattern

Tip: Do not memorize patterns by definition. Learn the problem first, then understand why the pattern exists.

---

11. Observability

Logging

- Structured Logging
- Centralized Logging
- Log Levels
- Correlation IDs

Metrics

- Metrics
- Latency Metrics
- Error Metrics
- Throughput Metrics
- p50
- p95
- p99

Distributed Tracing

- Distributed Tracing
- Trace IDs
- Context Propagation
- OpenTelemetry Basics

Reliability Metrics

- SLI
- SLO
- SLA
- Error Budgets
- Alerting

Tip: A production system is not truly reliable if you cannot detect when it is failing.

---

12. Security

Only learn security concepts directly relevant to designing backend systems.

Authentication & Authorization

- Authentication
- Authorization
- JWT
- OAuth 2.0
- OpenID Connect
- API Keys
- RBAC

System Security

- TLS
- Encryption in Transit
- Encryption at Rest
- Secrets Management
- Key Rotation
- Rate Limiting
- DDoS Protection

---

13. Technology Deep Dives

Do these only after understanding the general concepts.

Databases

- PostgreSQL
- DynamoDB
- MongoDB
- Cassandra

Caching

- Redis

Messaging

- Kafka
- RabbitMQ
- Amazon SQS

Search

- Elasticsearch / OpenSearch

Infrastructure

- Nginx
- Kubernetes Basics

---

14. System Design Practice

Beginner

- URL Shortener
- Rate Limiter
- Pastebin
- Notification System

Intermediate

- Chat System
- Instagram
- Twitter / X Feed
- File Storage
- Web Crawler
- E-Commerce System

Advanced

- YouTube
- Netflix
- WhatsApp

- Uber
- Google Drive
- Search Engine
- Distributed Cache
- Distributed Queue

For Every System Design

- Functional Requirements
- Non-Functional Requirements
- Capacity Estimation
- APIs
- Data Model
- High-Level Architecture
- Database Choice
- Caching
- Asynchronous Processing
- Scaling Strategy
- Partitioning
- Replication
- Consistency
- Reliability
- Security
- Observability
- Bottlenecks
- Tradeoffs
- Failure Scenarios

Core Questions For Every Topic

- What problem does this solve?
- Why does this problem exist?
- How does it work internally?
- What are the tradeoffs?
- When should I use it?
- When should I not use it?
- What happens at scale?
- What happens when it fails?
- What are the alternatives?

Golden Rule

Do not learn a topic just because it exists.

Learn it if it helps answer one of these questions:

- How do I scale this?
- How do I store this?
- How do components communicate?
- How do I maintain consistency?
- What happens when something fails?
- What bottleneck will appear?
- What tradeoff am I making?

Focus on understanding the problem first, then the solution, then the tradeoffs.

prompt-to-learn

Whenever I give you a **system design topic**, teach it to me in a way that builds deep intuition, not just theoretical knowledge.

Follow this learning style:

1. **Start with the problem**
    - What real-world problem does this concept solve?
    - Why did engineers need to invent it?
    - What would happen if we did not have it?
2. **Build intuition first**
    - Explain it simply and logically.
    - Use a realistic example from a modern tech system such as Netflix, WhatsApp, Uber, Amazon, Instagram, or a backend service.
    - Make me understand the thought process that leads naturally to the solution.
3. **Explain how it actually works**
    - Go step by step.
    - Explain the flow of data, requests, components, and decisions.
    - Use simple diagrams when useful.
4. **Use a concrete real-world example throughout**
    - Do not keep changing examples unnecessarily.
    - Prefer one realistic example that becomes more complex as the concept is explained.
5. **Explain tradeoffs deeply**
    - When should I use it?
    - When should I not use it?
    - What are the advantages and disadvantages?
    - What problems or limitations does it introduce?
    - What alternatives exist and why might I choose them?
6. **Connect it to system design**
    - Where does this appear in a real architecture?
    - What other components does it interact with?
    - What system design problems can it solve?
7. **Explain failures and edge cases**
    - What can go wrong?
    - What happens when a dependency fails?
    - How does the system recover?
    - What happens at scale?
8. **Make me think**
    - Do not immediately give me everything.
    - Occasionally ask me small questions to check whether I understand the reasoning.
    - If I have a wrong understanding, correct my intuition instead of simply giving the answer.
9. **End with strong retention**
    - Give me a concise mental model.
    - Summarize the most important ideas.
    - Give me a few memorable rules or shortcuts.

    - Include common interview questions or misconceptions if useful.

Important

- Prioritize **intuition over jargon**.
- Do not explain something just because it is technically correct. Explain **why it exists and why it works**.
- Do not give me an overwhelming wall of information at once.
- Go deep, but build complexity gradually.
- Assume I want to become genuinely strong at system design, not memorize interview answers.
- Relate concepts to each other whenever possible.
- If a topic has prerequisites, briefly connect them instead of assuming I already know everything.
- Use real engineering scenarios, realistic traffic, databases, caches, queues, services, and failures.
- Help me build a mental model that I can reuse in future system designs.

Most importantly: Make me understand the reasoning so deeply that I can derive the concept again even if I forget the definition.

Questions to ask before diving

You are an experienced FAANG system design interviewer conducting a interview for an SDE 1-2 role. I will design a system following my template, and you will evaluate me like a real interviewer.

Your role:

1. Ask clarifying questions when I make vague statements
2. Challenge my design choices with "what if" scenarios
3. Point out bottlenecks I missed
4. Ask me to deep dive into specific components
5. Be realistic - sometimes accept "good enough" answers, don't expect perfection
6. Give hints if I'm completely stuck (like a friendly interviewer would)
7. At the end, give honest feedback on what I did well and what I should improve

Interview flow:

1.As I go through each phase, ask probing questions

2.After high-level design, pick 1-2 areas and ask me to go deeper

3.Challenge me with scale/failure scenarios

Important: Don't let me skip the data model phase. If my database choice seems wrong, challenge it. Act like you're genuinely curious why I made certain decisions.

this is my template

Phase 1: Requirements (2-3 minutes)(both functional & non functional)

Ask only these critical questions:

- What are the core features? (pick 2-3 to focus on)
- Read-heavy or write-heavy?
- Scale estimate? (100 users, 1M users, or 1B users)
- Availability > Consistency, or opposite?

State your assumptions for everything else.

Phase 2: Capacity Planning (2 minutes)

Quick math only if scale is large:(based on read is to write)

- Storage: users × data per user
- Traffic: requests per second at peak
- Bandwidth: if media-heavy
- Memory: for caching

Skip this for small systems.

Phase 3: Data Model (5 minutes)

This is your most important phase.

Ask yourself:

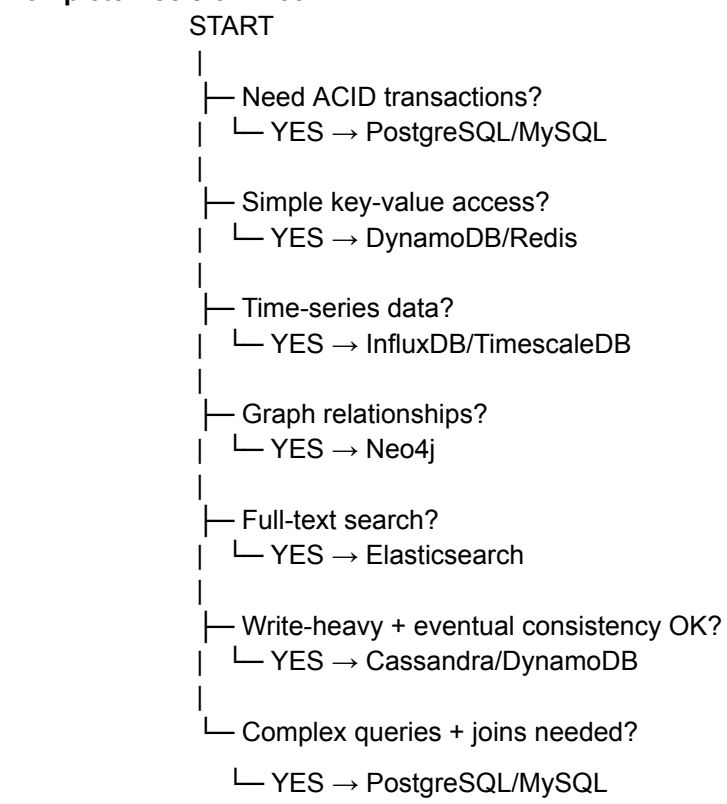
- What are my main entities? (User, Post, Comment, etc.)
- How will I query them?
    - By ID? → any DB works
    - By time/range? → time-series optimized
    - By relationships? → graph or relational
    - By search? → need search engine

Database decision tree:(read from page 5 for clarity)

Questions to ask:

1. **Traffic Pattern Analysis(read-heavy or write heavy or balanced)**
2. **data model type(key based,Queries with Joins, time based, relationships)**
3. **Consistency Requirements(strong or eventual)**

Complete Decision Tree



Define 3-4 tables with key fields. Show primary keys and important indexes.

Phase 4: API Design (3 minutes)

Define 3-4 key APIs:

- POST /tweets (user_id, content, media_urls)
- GET /timeline (user_id, page_token)

POST /follow (follower_id, followee_id)

Keep it simple. This shows you think API-first.

Phase 5: High-Level Design (10 minutes)

Draw the basic flow:

- Client → CDN (for static)
- → Load Balancer
- → App Servers (stateless)
- → Cache (Redis)
- → Primary DB

→ Replicas (read)

For each component, explain one sentence why it's there.

Add these only if relevant to your use case:

- Message Queue (if async processing needed)
- Object Storage (if media files)
- Search Service (if search is core feature)

Phase 6: Deep Dive (10 minutes)

Pick ONE or TWO based on what matters:

For read-heavy:

- Cache strategy (what to cache, TTL, invalidation)
- Read replicas (how to route, lag handling)

For write-heavy:

- Sharding strategy (by user_id, by time, consistent hashing)
- How to handle write conflicts

For real-time:

- WebSockets vs polling

- How to push updates

For analytics:

- How to aggregate without killing DB
- Batch processing vs stream processing

Phase 7: Bottlenecks & Trade-offs (5 minutes)

Mention these quickly:

- Single points of failure → add replicas
- Database bottleneck → sharding or read replicas
- Cache failure → fallback to DB (degraded performance)
- Consistency issues → mention eventual consistency trade-off

Don't go deep unless asked.

Questions to Ask Yourself (Mental Checklist)

During the interview, silently verify:

- Is this CRUD, feed-based, real-time, or analytics?
- What's the read/write ratio?
- Do I need strong consistency anywhere?
- What's the access pattern for my main entity?
- Where will the bottleneck be at 10x scale?

Grok-HLD

PHASE 0: Foundation

Before any problems:

1. **System Design Metrics** - Latency, throughput, availability, durability
2. **CAP Theorem** - Understand trade-offs deeply
3. **Horizontal vs Vertical Scaling**
4. **Back-of-envelope calculations** - Practice estimating storage/bandwidth

Time: 3-4 days

Without this, trade-offs in TinyURL (availability > consistency) won't make sense.

PHASE 1: Simple Systems

1. **URL Shortener (TinyURL)** - Basic scaling, hashing, databases
2. **Pastebin** - Reinforces key generation

PHASE 2: Core Patterns (Reordered)

3. **Typeahead Suggestion** - Trie, caching
4. **API Rate Limiter** - Token bucket, sliding window (moved here after understanding caching)
5. **Web Crawler** - BFS/DFS, distributed systems

PHASE 3: Real-time & Streaming

Add primer on:

- Message Queues (Kafka, RabbitMQ)
- Producer/Consumer pattern
- websockets

6. **Instagram** - Photo storage, CDN, background jobs
7. **Facebook Messenger** - WebSockets, real-time delivery
8. **YouTube/Netflix** - Video encoding, chunking, streaming

PHASE 4: Social Networks (Split Twitter)

9. **Twitter (Timeline)** - Fanout, feed caching, writes
10. **Dropbox** - File sync, chunking, conflict resolution

11. **Twitter Search** - Inverted indices, ranking (separate from timeline)

PHASE 5: Location-Based (Add Maps)

Optional but recommended: 12. **Google Maps** - Graph traversal, geohashing, tile caching

13. **Yelp** - QuadTree, spatial indexing
14. **Uber** - Dynamic updates, real-time matching

PHASE 6: Complex Systems

15. **Ticketmaster** - Concurrency, distributed locks, transactions
16. **Facebook Newsfeed** - Revisit all concepts: caching strategies, fanout models, pre-computation

At this stage, **actively connect back** to Instagram/Twitter patterns.

Key Adjustments Made

1. **Added Phase 0** - Foundation concepts must come first
2. **Moved Rate Limiter** - After understanding distributed counters/caching
3. **Split Twitter** - Timeline vs Search are different problem spaces
4. **Added streaming primer** - Message queues before media systems
5. **Added Maps** - Bridge between static (Yelp) and dynamic (Uber) geo systems
6. **Emphasized revisiting** - Phase 6 should consciously connect to earlier learnings

Critical Study Habit

For each problem, document:

- **Pattern used** (e.g., "fanout-on-write pattern")
- **Trade-off made** (e.g., "chose availability over consistency because...")
- **Reusable component** (e.g., "rate limiter can also throttle API requests")

DB

Database Concepts to Master for System Design

1. Foundations

- **Relational Model**
    - Tables, rows, columns, primary keys, foreign keys
    - Normalization (1NF, 2NF, 3NF, BCNF) vs Denormalization
- **SQL** basics: `SELECT`, `JOIN`, `GROUP BY`, indexes
- **Data Types** and constraints
- **ACID properties** (Atomicity, Consistency, Isolation, Durability)

---

2. Indexes & Query Optimization

- **B-Trees, B+ Trees(Data Structure Layer)**
- **Hash indexes(Data Structure Layer)**
- Covering indexes, composite indexes(Logical Usage Layer)
- Clustered vs Non-clustered indexes(Logical Usage Layer)
- Query execution plans (how DB runs queries)
- Common pitfalls: full table scan, index scan vs index seek

---

3. Transactions & Concurrency

- Transaction isolation levels:
    - Read uncommitted, Read committed, Repeatable read, Serializable

- Problems: Dirty reads, Phantom reads, Lost updates
- **MVCC (Multi-Version Concurrency Control)**
- Locking mechanisms: row locks, table locks, optimistic vs pessimistic locking

---

4. Partitioning & Sharding

- Horizontal vs Vertical partitioning
- Range, List, Hash partitioning
- Sharding strategies:
    - Range-based
    - Hash-based
    - Directory/Lookup service
- Tradeoffs: rebalancing shards, hotspots, cross-shard joins

---

5. Replication & High Availability

- **Replication types**:
    - Master-slave
    - Master-master
    - Quorum-based (Raft, Paxos)
- **Synchronous vs Asynchronous replication**
- Failover strategies
- Read replicas (for scaling reads)

---

6. Consistency & Distributed Systems

- **CAP Theorem** (Consistency, Availability, Partition tolerance)
- **PACELC Theorem** (tradeoff between Latency and Consistency under normal conditions)
- Eventual consistency
- Strong vs weak vs causal consistency
- Conflict resolution (last write wins, vector clocks, CRDTs)

---

7. Storage & Internals

- How data is stored on disk (pages, extents)
- Write-ahead log (WAL)
- Buffer pool, caching, checkpoints
- SSD vs HDD optimization
- LSM trees (used in Cassandra, RocksDB)

---

8. NoSQL Paradigms

- **Key-Value Stores** (Redis, DynamoDB)
- **Document Stores** (MongoDB, Couchbase)
- **Column Stores** (Cassandra, HBase)
- **Graph Databases** (Neo4j, ArangoDB)
- When to choose SQL vs NoSQL

---

9. Advanced Concepts for System Design

- **CQRS (Command Query Responsibility Segregation)**
- **Event Sourcing** (append-only logs)
- Materialized views
- Read-write separation
- Caching strategies: write-through, write-back, write-around
- **Schema design for scale** (e.g., wide tables in NoSQL vs normalized tables in RDBMS)

---

10. Distributed Transactions

- Two-phase commit (2PC)
- Three-phase commit (3PC)
- Sagas (long-running workflows with compensating transactions)

---

11. Specialized DBs & Modern Trends

- **Time-series databases** (InfluxDB, TimescaleDB)
- **Search engines** (Elasticsearch, Solr)
- **NewSQL** (CockroachDB, YugabyteDB)
- **Vector databases** (Pinecone, Weaviate — for AI/ML use cases)

---

12. DB in System Design Interviews

- When to pick SQL vs NoSQL

- Tradeoffs between sharding vs replication
- Designing for high availability & failover
- Schema design for specific use cases (social media, e-commerce, logging, financial systems)
- Scaling read-heavy vs write-heavy systems

WHEN TO CHOOSE WHICH DB:

Database Selection Guide

Step 1: Traffic Pattern Analysis

Read-Heavy (90%+ reads)

- **Best choice:** SQL with read replicas
- **Why:** Caching handles most reads, SQL gives you flexibility
- **Setup:**
  - 1 Primary (writes)
  - Multiple Read Replicas (reads)
  - Redis/Memcached in front
- **Examples:** E-commerce product catalog, blog platforms

Write-Heavy (60%+ writes)

- **Best choice:** NoSQL (Cassandra, DynamoDB)
- **Why:** Horizontal scaling for writes, eventual consistency acceptable
- **Setup:**
  - Partition(in nosql partition means sharding) by user_id or time
  - Multiple nodes share write load
- **Examples:** Logging systems, IoT data ingestion, metrics collection

Balanced (50-50 read/write)

- **Best choice:** SQL with good indexing OR NoSQL if scale is massive
- **Why:** Need flexibility but also performance
- **Setup:**
  - SQL: Postgres with proper indexes + read replicas
  - NoSQL: If you expect 10M+ users
- **Examples:** Social media apps, messaging platforms

Step 2: Data Model Complexity

Simple Key-Value Access

- User by user_id
- Session by session_id
- Cache-like operations → **DynamoDB, Redis, Cassandra**

Complex Queries with Joins

- Get user's posts with comments and likes
- Multiple table joins needed
- Need transactions → **PostgreSQL, MySQL**

Time-Based Queries

- Get all events in last 24 hours
- Metrics, logs, sensor data
- Aggregate by time windows → **InfluxDB, TimescaleDB, Cassandra** (with time-based partition)

Graph Relationships

- Friends of friends
- Recommendation engines
- Social network traversals → **Neo4j, DGraph**

Full-Text Search

- Search tweets by keywords
- Fuzzy matching, autocomplete → **Elasticsearch, Solr**

Step 3: Consistency Requirements

Strong Consistency Needed

- Banking transactions
- Inventory management
- Booking systems (no double booking) → **PostgreSQL, MySQL** (ACID transactions)

Eventual Consistency OK

- Social media likes/views count
- Analytics dashboards
- Feed generation → **Cassandra, DynamoDB, MongoDB**

Step 4: Scale Considerations

Small Scale (< 100K users)

- **Use:** PostgreSQL
- **Why:** Simplest, handles everything, easy to manage

Medium Scale (100K - 10M users)

- **Use:** PostgreSQL with read replicas + Redis cache
- **Why:** Can still handle with vertical + horizontal read scaling

Large Scale (10M+ users)

- **Use:** NoSQL for hot data, SQL for cold data
- **Why:** Need horizontal write scaling
- **Pattern:**
    - Cassandra/DynamoDB for recent/active data
    - PostgreSQL for long-term/analytical queries
    - S3 for old data (archive)

Complete Decision Tree

```
START
|
├─ Need ACID transactions?
|  └─ YES → PostgreSQL/MySQL
|
├─ Simple key-value access?
|  └─ YES → DynamoDB/Redis
|
├─ Time-series data?
|  └─ YES → InfluxDB/TimescaleDB
|
├─ Graph relationships?
|  └─ YES → Neo4j
|
├─ Full-text search?
|  └─ YES → Elasticsearch
|
├─ Write-heavy + eventual consistency OK?
|  └─ YES → Cassandra/DynamoDB
|
└─ Complex queries + joins needed?
   └─ YES → PostgreSQL/MySQL
```

Quick Reference Table

Use Case	Read/Write Pattern	Best DB	Reasoning
URL Shortener	Read-heavy (95:5)	PostgreSQL + Redis	Simple schema, cache handles reads
Twitter Feeds	Read-heavy (90:10)	Cassandra + Redis	Scale writes, eventual consistency OK

Banking	Balanced	PostgreSQL	ACID critical, transactions needed
Analytics/Logs	Write-heavy (10:90)	Cassandra/InfluxDB	High write throughput, time-based
LinkedIn Network	Balanced	Neo4j + PostgreSQL	Graph for connections, SQL for profiles
E-commerce Search	Read-heavy	Elasticsearch + PostgreSQL	Search primary, SQL for transactions
Chat Messages	Write-heavy (40:60)	Cassandra/MongoDB	High writes, simple queries by user/time
Gaming Leaderboard	Balanced	Redis (sorted sets)	In-memory speed, simple operations

---

Common Interview Mistakes to Avoid

❌ **Don't say:** "I'll use MongoDB because it's NoSQL and scalable" ✅ **Say instead:** "I'll use PostgreSQL initially because our data model has relationships and we need ACID. If we hit 10M+ users, we can shard or move hot data to Cassandra."

❌ **Don't say:** "NoSQL is always better for scale" ✅ **Say instead:** "NoSQL helps with horizontal write scaling, but we lose ACID guarantees and join capabilities. For this use case, PostgreSQL with read replicas can handle our scale."

❌ **Don't say:** "We need Cassandra because it's what big companies use" ✅ **Say instead:** "Cassandra makes sense here because we have high write volume, eventual consistency is acceptable, and we're querying by time ranges which Cassandra handles well."

1. Query Design & Schema Modeling (Very Underrated)

Most engineers focus on scaling before mastering modeling.

You should deeply master:

- Modeling many-to-many relationships
- Handling soft deletes vs hard deletes
- Handling large JSON columns vs normalized tables
- Choosing surrogate key vs natural key
- UUID vs auto increment tradeoffs
- Designing idempotent schemas
- Handling hot keys

Without this, everything else breaks at scale.

---

2. Deep Index Internals

You mentioned B Trees. Go deeper:

- How page splits work
- Fill factor
- Why random UUID hurts clustered indexes
- How composite index ordering works
- Selectivity and cardinality
- Why index on low cardinality column is useless
- How covering index avoids table lookup

Also understand why write-heavy systems suffer with too many indexes.

---

3. Execution Engine & Optimizer Internals

Very few engineers understand this.

You should know:

- Cost based optimizer
- Nested loop join vs hash join vs merge join
- Why JOIN order matters
- Why LIMIT changes execution plan
- How statistics affect performance
- Why stale stats cause bad plans

This separates mid engineers from senior DB engineers.

---

4. Memory Architecture

You mentioned buffer pool, but expand it:

- Shared buffers vs OS cache
- WAL buffer
- Checkpoint tuning
- Vacuum process in PostgreSQL
- Autovacuum issues
- Memory fragmentation

If you don’t understand memory, you cannot debug production DB issues.

---

5. Storage Engines (Critical Gap)

Different engines behave differently.

For example:

- InnoDB in MySQL
- Heap + MVCC in PostgreSQL

- LSM tree in Cassandra
- B Tree storage in traditional RDBMS

You must understand write amplification in LSM trees.
Also compaction strategies:

- Size tiered
- Leveled compaction

---

6. Real World Failure Modes

This is missing from your list and extremely important.

- Split brain
- Replica lag
- Thundering herd problem
- Hot partition
- Write amplification
- Disk saturation
- Lock contention
- Deadlocks and how DB resolves them

System design interviews love these.

---

7. Observability & Production Debugging

Master these:

- Slow query logs
- EXPLAIN ANALYZE
- Lock wait graphs

- Deadlock logs
- Replication lag monitoring
- Metrics: QPS, TPS, P95 latency

Without this, you cannot operate databases.

---

8. Backup & Disaster Recovery

You didn’t include this, but in production this is critical.

- Full backup vs incremental backup
- Point in time recovery
- WAL archiving
- RPO and RTO
- Cross region replication

Many system design candidates forget this completely.

9. Real Database Examples

1. PostgreSQL

- MVCC implementation
- WAL
- Vacuum
- Index types
- Partitioning

2.Apache Cassandra

- Consistent hashing
- Partition key design
- LSM tree
- Compaction
- Tunable consistency

3. Redis

- In memory data structures
- Persistence models RDB vs AOF
- Eviction policies
- Clustering

Everything Python

https://chatgpt.com/g/g-p-6a015e9bc2048191acebc0acd27a8651-system-design/c/6a96a188-6100-83ee-a6ca-029e90eeea05

PHASE 0: HOW PYTHON ACTUALLY WORKS (FOUNDATION LAYER)

---

1. Execution Model
   - Source → bytecode → .pyc
   - dis module (inspect bytecode)
   - Python Virtual Machine
   - Stack frames
   - Call stack mechanics

2. CPython Architecture
   - eval loop
   - PyObject structure (ob_refcnt, ob_type)
   - Memory allocator
   - Reference counting
   - Cyclic garbage collector

3. Python Data Model Overview
   - Everything is an object
   - Type objects
   - How attribute lookup works
   - Method binding mechanics
   - Descriptor protocol (preview)

4. Frame Objects
   - What a frame contains
   - f_locals, f_globals
   - sys._getframe()

5. Implementations
   - CPython
   - PyPy
   - Jython
   - MicroPython

6. GIL Deep Understanding
   - Why it exists
   - Bytecode atomicity
   - CPU bound vs IO bound
   - When GIL is released

PHASE 1: CORE LANGUAGE FUNDAMENTALS

---

1. Data Types and Memory Model
   - int, float, bool
   - None
   - str (immutability, interning)
   - bytes vs bytearray
   - list, tuple
   - dict (hash table internals)
   - set, frozenset

2. Object Identity
   - id()
   - is vs ==
   - hash()
   - Hash contract (eq and hash relationship)
   - Why mutable objects should not be hashable

3. Namespaces and Scope
   - LEGB rule
   - locals()
   - globals()
   - Name binding vs mutation

4. Control Flow
   - if, while, for
   - else in loops
   - match statement

5. Functions Deeply
   - Positional, keyword-only
   - Mutable default argument trap
   - *args and **kwargs
   - Closures
   - nonlocal
   - Annotations

6. Comprehensions
   - List, dict, set
   - Generator expressions
   - Scoping behavior inside comprehensions

7. Exceptions
   - try/except/else/finally
   - Custom exceptions
   - Exception chaining
   - raise from

PHASE 2: OBJECT MODEL AND OOP (DATA MODEL DEEP DIVE)
---

1. Class Mechanics
   - Class vs instance attributes
   - __init__
   - __new__
   - __del__
   - __dict__

2. Dunder Methods
   - __str__, __repr__
   - __eq__, __hash__
   - __len__
   - __iter__, __next__
   - __call__
   - __enter__, __exit__
   - __getattr__, __getattribute__

3. Method Resolution Order
   - C3 linearization
   - super() internals
   - Multiple inheritance edge cases

4. Descriptors (Deep)
   - __get__, __set__, __delete__
   - How properties work
   - How methods are descriptors

5. Metaclasses
   - type
   - Custom metaclass basics

6. Abstract Base Classes
   - abc module
   - @abstractmethod

7. Data Containers Comparison (Important in Interviews)

   dataclasses
   - How they are generated
   - frozen
   - slots
   - eq, order behavior

   attrs

- Validators
- Converters
- Performance aspects

pydantic
- Validation model
- Runtime parsing
- Use cases in APIs
- Performance tradeoffs

When to use:
- dataclasses for simple domain models
- attrs for advanced validation without heavy runtime cost
- pydantic for IO boundary validation (APIs, config)

PHASE 3: FUNCTIONAL AND ADVANCED FEATURES
---

1. First Class Functions
2. lambda
3. map, filter, reduce

4. functools Deep Dive
- functools.lru_cache
- functools.cache
- functools.partial
- functools.wraps
- reduce

5. Decorators
- Function decorators
- Class decorators
- Parameterized decorators

6. Iterators and Generators
- Iterator protocol
- yield
- yield from
- send, throw
- Generator internals

7. Context Managers
- with statement
- contextlib
- Custom context manager
- Async context manager

PHASE 4: MODULES, IMPORT SYSTEM, PACKAGING
---

1. Import System Internals
   - sys.modules
   - Import caching
   - Circular imports
   - Module spec

2. Packages
   - __init__.py
   - Relative vs absolute imports

3. Virtual Environments
   - venv
   - Dependency resolution

4. Packaging
   - pyproject.toml
   - Wheels
   - Entry points
   - Versioning strategies

PHASE 5: CONCURRENCY AND PARALLELISM
---

1. Threading
   - threading module
   - Lock, RLock
   - Condition
   - Event
   - Semaphore

2. concurrent.futures (Very Practical)
   - ThreadPoolExecutor
   - ProcessPoolExecutor
   - submit vs map
   - as_completed
   - Exception handling in futures
   - When to choose it over raw threading

3. Multiprocessing
   - fork vs spawn
   - IPC
   - Shared memory

4. Asyncio
  - Event loop
  - Coroutines
  - Task
  - Future
  - gather
  - Cancellation
  - Async context managers

5. Choosing the Right Model
  - CPU bound vs IO bound
  - Scaling strategies

PHASE 6: MEMORY AND PERFORMANCE
---

1. Memory Management
  - Reference counting
  - Garbage collector generations
  - gc module

2. Performance Profiling
  - cProfile
  - timeit
  - memory_profiler
  - tracemalloc

3. Big O of Builtins
  - dict complexity
  - list complexity

4. Optimizations
  - __slots__
  - Generators
  - Avoiding temporary allocations
  - C extensions overview
  - Cython overview

PHASE 7: TYPE HINTS AND STATIC ANALYSIS
---

1. typing module
  - Union
  - Optional
  - Callable
  - Generics

  - TypeVar
  - Protocol
  - TypedDict

2. mypy basics
3. Runtime vs static typing tradeoffs

PHASE 8: DEBUGGING AND INTROSPECTION (MISSING BUT CRITICAL)
---

1. Reading Tracebacks Properly
  - Stack trace structure
  - Chained exceptions

2. pdb
  - breakpoints
  - stepping
  - inspecting variables

3. breakpoint()
4. traceback module
5. inspect module
6. logging module deep dive

PHASE 9: TESTING AND CODE QUALITY
---

1. unittest
2. pytest
3. Fixtures
4. Mocking
5. Property based testing
6. Coverage
7. Linting
  - flake8
  - black
  - isort

PHASE 10: STANDARD LIBRARY MASTERY

High impact modules:

- collections
- itertools

- functools
- datetime
- pathlib
- os
- sys
- subprocess
- logging
- argparse
- json
- pickle
- re
- heapq
- bisect
- threading
- concurrent.futures

Python

PYTHON INTERNALS REVISION

Q1. When you run `python app.py`, explain the full execution pipeline.

Answer:

1. Python reads the source file.
2. The **tokenizer** converts source code into tokens.
3. The **parser** builds an **AST** (Abstract Syntax Tree).
4. The AST is compiled into **Python bytecode**.
5. The bytecode is stored in **code objects**.
6. For imported modules, Python writes bytecode into **`.pyc` files** in the `__pycache__` directory.
7. The **CPython interpreter** executes the bytecode using the **Python Virtual Machine (PVM)**.
8. The **evaluation loop** reads each instruction and executes it.

Important Notes:

- **`.pyc` files contain:** Magic number, Timestamp/hash of source code, Compiled bytecode.
- **If `.pyc` is deleted:** Python recompiles the module during the next import.
- **Why Python uses bytecode:** Platform independent, Simplifies interpreter implementation, Allows caching of compiled modules.

Q2. What is the Python Virtual Machine?

Answer:

The **Python Virtual Machine (PVM)** is the runtime component inside CPython responsible for executing Python bytecode.

Characteristics:

- **Stack-based** architecture
- Executes bytecode instructions sequentially
- Manages **stack frames** and function calls
- Uses an **operand stack** for computations

Feature	JVM	PVM
Compilation	Often uses JIT compilation	Mostly interprets bytecode

Typing	Statically typed language	Dynamic typing
Source to Bytecode	Compiles Java source separately using `javac`	Compilation happens automatically during execution

The interpreter loop exists in CPython source code: `Python/ceval.c`

Q3. Explain how Python executes this function internally:

```
def add(a, b):
    return a + b

add(2,3)
```

Execution Steps:

1. Python compiles the function into a **code object**.
2. When `add(2,3)` is called, CPython creates a **stack frame**.
3. The frame is pushed onto the **call stack**.
4. Arguments are stored as **local variables** in the frame.
5. The evaluation loop begins executing bytecode instructions.

Typical Bytecode:

Instruction	Action
`LOAD_FAST a`	Pushes local variable `a` onto operand stack.
`LOAD_FAST b`	Pushes local variable `b` onto operand stack.
`BINARY_ADD`	Pops two operands from stack, performs addition, pushes result.

`RETURN_VALUE`	Pops result and returns it to caller.

After completion: The frame is removed from the call stack.

Q4. What is a stack frame?

Answer:

A **stack frame** represents the execution context of a function call.

Each frame contains:

- **Local variables**
- Global variables reference
- Builtins reference
- **Instruction pointer** (current bytecode index)
- **Value stack** (operand stack)
- Code object being executed
- Link to the **previous frame (caller)**

Frames together form the **call stack**.

Example: `main()` -> `foo()` -> `bar()`. Each call creates a new frame.-----5. BYTECODE INSPECTION

Q5. How can you inspect Python bytecode?

Answer:

Python provides the **`dis` module**.

Example:

```
import dis


def add(a, b):
    return a + b


dis.dis(add)
```

Example output:

LOAD_FAST 0 (a)

LOAD_FAST 1 (b)

BINARY_ADD

RETURN_VALUE

Meaning:

- `LOAD_FAST`: Push local variable onto stack
- `BINARY_ADD`: Pop two values, perform addition
- `RETURN_VALUE`: Return top value from stack

Q6. What is the CPython evaluation loop?

Answer:

The **evaluation loop** is the core interpreter loop responsible for executing Python bytecode instructions.

Location in CPython source code: `Python/ceval.c`

The loop repeatedly performs:

1. Fetch next bytecode instruction
2. Decode the instruction
3. Execute the instruction

Pseudo representation:

while true:

opcode = fetch_instruction()

execute(opcode)

This is why CPython is called a bytecode interpreter.-----7. PYOBJECT STRUCTURE

Q7. What is PyObject?

Answer:

Every Python object in CPython begins with a `PyObject` **structure**.

Simplified structure:

typedef struct {

Py_ssize_t ob_refcnt;

PyTypeObject *ob_type;

} PyObject;

Fields:

- **`ob_refcnt`**: Number of references pointing to the object. (Used for memory management)
- **`ob_type`**: Pointer to the object's type (class).

This design allows CPython to treat all objects uniformly.

Q8. Explain reference counting in Python.

Answer:

Python uses **reference counting** for memory management.

Every object maintains a count of how many references point to it.

Example:

a = [] # ob_refcnt increases

b = a # ob_refcnt increases

del a # ob_refcnt decreases

When reference count becomes zero: The object is immediately deallocated from memory.-----9. CYCLIC GARBAGE COLLECTION

Q9. Why does Python also have a cyclic garbage collector?

Answer:

Reference counting cannot detect circular references.

Example (Circular Reference):

class A: pass

a = A()

b = A()

a.ref = b

b.ref = a

The reference count for a and b never reaches zero, even if no external variables reference them.

Solution:

Python uses a **cyclic garbage collector** that periodically:

- scans objects
- detects unreachable cycles
- frees them

-----10. PYTHON MEMORY ALLOCATION

Q10. How does Python allocate memory?

Answer:

CPython uses a specialized memory allocator called **`pymalloc`** for small objects.

Memory structure hierarchy:

- **Arena**: Large memory region (~256 KB)
- **Pool**: Contains blocks of fixed size
- **Block**: Actual memory used for objects

Structure: Arena -> Pools -> Blocks

Benefits: Faster allocations, Reduced fragmentation, Efficient reuse of memory.-----11. EVERYTHING IS AN OBJECT

Q11. What does "everything is an object" mean?

Answer:

In Python, all data, including:

- **integers**
- **functions**
- **classes**
- **modules**
  ...are implemented as objects.

Example: `x = 10; print(type(x))`

Internally each object contains: **reference count**, **type information**, and **actual value**.

Even classes themselves are objects created by the metaclass `type`.-----12. ATTRIBUTE LOOKUP

Q12. How does attribute lookup work (e.g., `obj.attr`)?

Lookup order (Simplified):

1. **Data descriptors** in the class (e.g., properties with `__set__`)
2. Instance `__dict__`
3. **Non-data descriptors** in the class (e.g., methods)
4. Class attributes
5. Parent classes via **MRO** (Method Resolution Order)

Descriptors can override attribute access behavior.-----13. METHOD BINDING

Q13. Why does Python automatically pass `self`?

Example:

```
class A:
    def f(self):
        pass
a = A()
a.f()
```

Answer:

Functions inside classes are **descriptors**.

When accessed through an instance (`a.f`), Python converts the function into a **bound method**.

Bound method = function + instance

Calling `a.f()` is equivalent to: `A.f(a)`. The instance `a` is automatically passed as the first argument (`self`).-----14. DESCRIPTOR PROTOCOL

Q14. What is a descriptor?

Answer:

A **descriptor** is any object implementing one or more of the following methods:

- `__get__(self, obj, type)`

- `__set__(self, obj, value)`
- `__delete__(self, obj)`

Descriptors allow **customization of attribute access**.

Example: `property` is implemented using the descriptor protocol.-----15. FRAME OBJECTS

Q15. What is a frame object?

Answer:

A **frame object** represents the **execution state of a function**.

Important fields:

- `f_locals`: Dictionary of local variables.
- `f_globals`: Global namespace reference.
- `f_code`: Code object being executed.
- `f_back`: Reference to the previous frame.

Frames together form the **call stack**.-----16. `sys._getframe()`

Q16. What does `sys._getframe()` do?

Answer:

`sys._getframe()` returns the **current stack frame**.

Uses: debugging, logging frameworks, and stack inspection.-----17. CPYTHON VS PYPY

Q17. Difference between CPython and PyPy.

Answer:

	CPython	**PyPy**
Type	**Reference**	Alternative

	implementation	implementation
Language	Written in C	Written in RPython
Execution	**Interprets bytecode**	Includes a **JIT compiler**

The **JIT (Just-In-Time)** in PyPy dynamically compiles frequently executed code into machine code, improving performance.-----18. WHY JYTHON CANNOT USE NUMPY

Q18. Why can't Jython run many C extensions (like NumPy)?

Answer:

Jython runs on the **Java Virtual Machine (JVM)** and uses **Java objects** instead of CPython objects.

Libraries like NumPy rely heavily on the **CPython C API**.

Since Jython does not implement the CPython C API, C extensions cannot run there.-----Python Concurrency: The Global Interpreter Lock (GIL)-----19. GLOBAL INTERPRETER LOCK

Q19. What problem does the GIL solve?

Answer:

The **Global Interpreter Lock (GIL)** ensures that **only one thread executes Python bytecode at a time** within CPython.

It protects:

- **Reference counting** (preventing race conditions on `ob_refcnt`)
- **Object memory safety**
- **Interpreter state**

Without the GIL, **race conditions** could corrupt memory.-----20. IS PYTHON SINGLE THREADED

Q20. Is Python single threaded?

Answer:

Python supports multithreading.

However, in CPython, only one thread can execute Python bytecode at a time due to the **GIL**.

Threads still provide concurrency for **IO-bound** tasks (where the thread releases the GIL while waiting for IO).-----21. CPU BOUND THREADS

Q21. Why do threads not speed up CPU bound tasks?

Answer:

CPU bound threads repeatedly compete for the **GIL**.

Since only one thread can execute Python bytecode at a time, threads effectively run **sequentially** rather than in true parallel.

Multiprocessing is used instead to utilize multiple CPU cores for CPU-bound tasks.-----22. WHEN PYTHON RELEASES THE GIL

Q22. When does Python release the GIL?

Answer:

The GIL is released during **blocking operations** such as:

- **File IO**
- **Network IO**
- **Database queries**
- **Sleep operations**
- **Long running C extension computations**

This allows other threads to execute while one is blocked.-----23. THREAD EXAMPLE

Q23. Why does this not run faster?

```
import threading


def work():
    for i in range(10**8):
        pass


t1 = threading.Thread(target=work)
t2 = threading.Thread(target=work)
# ... start and join threads ...
```

Answer:

Both threads are **CPU-bound** and repeatedly **acquire and release the GIL**.

Since only one thread can execute Python bytecode at a time, they run **sequentially** rather than in parallel, offering no performance speedup over a single thread.-----24. NUMPY PARALLELISM

Q24. How does NumPy achieve parallelism despite the GIL?

Answer:

NumPy performs heavy computations in optimized **C code**.

During these operations, it explicitly **releases the GIL**.

This allows multiple CPU cores to execute the native C code in parallel, outside of the Python interpreter's GIL restriction.-----25. REMOVING THE GIL

Q25. What challenges arise if the GIL is removed?

Answer:

Removing the GIL introduces multiple issues:

- **Race conditions** in reference counting
- **Memory corruption risks** (due to unprotected shared object memory)
- Requirement for **fine-grained locking** (which is complex and error-prone)
- **Slower single-thread performance** (due to locking overhead)
- Existing **C extensions** depending on the GIL would break

These challenges make removing the GIL extremely difficult.

How did Python remove the GIL?

Answer:

Python made the GIL optional using atomic reference counting, immortal objects, and fine grained locks to maintain thread safety. However this introduces overhead, so single

threaded performance may decrease slightly while multithreaded workloads benefit significantly.

PHASE 1 — CORE LANGUAGE FUNDAMENTALS
INTERVIEW REVIEW + STRONG ANSWERS
==========

QUESTION 1
How Python stores large integers internally

Answer:

Python integers are implemented using the PyLongObject structure in CPython.

Unlike fixed-size integers in languages like C, Python integers are arbitrary precision. This means Python can represent very large numbers without overflow.

Internally a PyLongObject contains:
- reference count
- pointer to type
- size field (stores sign and number of digits)
- array of digits

Each digit represents a value in base 2^30 on 64-bit systems.

Example conceptually:

12345678901234567890

is stored internally as an array of digits.

Small integers from -5 to 256 are cached by CPython to improve performance since they are frequently used.

QUESTION 2
Difference between bytes and bytearray

Answer:

bytes
- immutable sequence of bytes
- hashable

- can be used as dictionary keys

Example:

b = b"hello"

bytearray
- mutable sequence of bytes
- not hashable

Example:

b = bytearray(b"hello")
b[0] = 72

Key difference:
bytes are immutable while bytearray is mutable.

Network protocols often use bytes because immutable data is safer and hashable.

QUESTION 3
Why Python strings are immutable

Answer:

Python strings are immutable for several reasons:

1. Memory efficiency
Immutable objects can be safely shared between variables.

2. Hashability
Strings can be used as dictionary keys.

3. String interning
Python can reuse identical strings in memory.

4. Thread safety
Immutable objects avoid race conditions.

Example:

a = "hello"
b = "hello"

Python may reuse the same object in memory.

QUESTION 4
Why this sometimes prints True

```
a = "hello"
b = "hello"

print(a is b)
```

Answer:

Python performs string interning, which means identical immutable strings may share the same memory object.

This usually happens for:
- short strings
- identifiers
- compile time constants

Because of this optimization, both variables may reference the same object.

QUESTION 5
Internal structure of Python lists

Answer:

A Python list is implemented as a dynamic array of pointers to objects.

Example list:

[1, 2, 3]

Internally:

[ptr → 1][ptr → 2][ptr → 3]

Important properties:

Append is fast because Python overallocates memory to reduce resizing operations.

Approximate growth strategy:

new_size = old_size * 1.125 + constant

Insert at the beginning is slow because all elements must shift to the right.

QUESTION 6
Tuple vs List internally

Answer:

List
- mutable
- dynamic size
- slightly larger memory footprint

Tuple
- immutable
- fixed size
- smaller memory footprint
- faster iteration

Tuples are hashable if all their elements are hashable, allowing them to be used as dictionary keys.

The performance advantage comes from immutability and simpler structure.

QUESTION 7
How Python dictionaries work internally

Answer:

Python dictionaries use a hash table.

Steps during insertion:

1. Compute hash of key

hash(key)

2. Use the hash to determine the index in the table.

3. If a collision occurs, Python uses open addressing with probing.

4. The key-value pair is stored in the hash table.

Average complexity of operations:

Lookup: O(1)
Insertion: O(1)
Deletion: O(1)

QUESTION 8
Why dictionary contains one key

Example:

```
d = {}
d[True] = "yes"
d[1] = "one"

print(d)
```

Answer:

True and 1 are considered equal in Python.

```
True == 1
hash(True) == hash(1)
```

Since dictionary keys must be unique, Python treats them as the same key and overwrites the value.

Final dictionary:

```
{True: 'one'}
```

QUESTION 9
Difference between set and frozenset

Answer:

set
- mutable
- cannot be used as dictionary key

frozenset
- immutable
- hashable
- can be used as dictionary key or element of another set

QUESTION 10
What id() returns

Answer:

In CPython, id() returns the memory address of the object.

Example:

```
x = 10
print(id(x))
```

However Python only guarantees that id is unique during the lifetime of the object.

After the object is destroyed, the same id may be reused.

QUESTION 11
Difference between is and ==

Answer:

is
Checks object identity (same memory location).

==
Checks value equality using the __eq__ method.

Example:

```
a = [1,2]
b = [1,2]

a == b   True
a is b   False
```

The operator is should be used for comparisons with:

None
True
False

QUESTION 12
What hash() is used for

Answer:

hash() computes an integer hash value used in hash-based data structures.

Used in:
- dictionaries
- sets
- frozensets

Objects used as keys must be hashable.

QUESTION 13
Hash contract between __eq__ and __hash__

Answer:

Hash contract rule:

If

a == b

then

hash(a) == hash(b)

However the reverse is not required.

hash(a) == hash(b) does not mean a == b.

Breaking this contract can cause dictionaries and sets to behave incorrectly.

QUESTION 14
LEGB rule

Answer:

Python variable lookup follows the LEGB rule:

L → Local scope
E → Enclosing scope
G → Global scope
B → Built-in scope

Example:

```
x = 10

def outer():
    x = 20

    def inner():
        print(x)
```

The inner function finds x in the enclosing scope.

QUESTION 15
Name binding vs mutation

Answer:

Binding:

```
x = [1,2]
```

A name is bound to an object.

Rebinding:

```
x = [3,4]
```

The name now points to a different object.

Mutation:

```
x.append(3)
```

The same object is modified.

QUESTION 17
Loop else clause

Answer:

The else block executes only if the loop finishes normally and is not terminated by a break.

Example:

```
for i in range(5):
    if i == 3:
        break
else:
    print("done")
```

Since break occurs, the else block does not execute.

QUESTION 19
Function parameter types

Example:

```
def f(a, b, /, c, *, d):
```

Meaning:

/ indicates positional-only parameters.

Parameters before / must be passed positionally.

* indicates keyword-only parameters.

Parameters after * must be passed using keywords.

QUESTION 20
Mutable default argument trap

Example:

```
def f(x=[]):
    x.append(1)
    return x

print(f())
print(f())
print(f())
```

Output:

```
[1]
[1,1]
[1,1,1]
```

Reason:

Default arguments are evaluated once when the function is defined, not each time the function is called.

Correct pattern:

```
def f(x=None):
    if x is None:
        x = []
```

QUESTION 21
Closures

Answer:

A closure is a function that remembers variables from its enclosing scope.

Example:

```
def outer():
    x = 10

    def inner():
        return x

    return inner
```

Even after the outer finishes execution, the inner still remembers x.

QUESTION 22
nonlocal keyword

Answer:

The nonlocal keyword allows modifying variables in the enclosing scope.

Example:

```
def outer():
    x = 10

    def inner():
        nonlocal x
        x += 1
```

Without nonlocal Python would create a new local variable.

QUESTION 25
List comprehension vs generator expression

Answer:

List comprehension:

```
[x*x for x in range(10)]
```

Creates the entire list in memory.

Generator expression:

```
(x*x for x in range(10))
```

Generates values lazily one at a time.

Generators use significantly less memory.

QUESTION 26

Comprehension scope behavior

Example:

```
x = 10
lst = [x for x in range(5)]
print(x)
```

Output:

10

Explanation:

In Python 3, variables inside list comprehensions have their own scope and do not overwrite the outer variable.

PHASE 2 — OBJECT MODEL AND OOP (DATA MODEL DEEP DIVE)
INTERVIEW REVIEW + STRONG ANSWERS
==========

QUESTION 1
Class attributes vs Instance attributes

Answer:

Class attributes belong to the class and are shared by all instances.

Instance attributes belong to a specific object instance.

Example:

```
class A:
    x = 10   # class attribute

a = A()
b = A()

a.y = 20    # instance attribute
```

Where they are stored:

Class attributes:
Stored inside the class dictionary

```
A.__dict__
```

Instance attributes:
Stored inside the instance dictionary

a.__dict__

Attribute lookup order:

obj.attr lookup happens in this order:

1. Data descriptor in class
2. Instance __dict__
3. Non-data descriptor in class
4. Class __dict__
5. Parent classes via MRO

QUESTION 2
What __init__ actually does

Answer:

__init__ initializes an already created object.

It sets the initial state of the instance.

Important points:

__init__ is NOT the constructor.

Object creation happens in:

__new__()

Sequence during object creation:

1. __new__ creates the object
2. __init__ initializes the object

Example:

```
class A:

    def __new__(cls):
        print("creating object")
        return super().__new__(cls)
```

```
    def __init__(self):
        print("initializing object")
```

If __init__ returns anything other than None, Python raises a TypeError.

QUESTION 3
Purpose of __new__

Answer:

__new__ is responsible for creating and returning a new object.

Signature:

def __new__(cls, ...)

It returns the instance that will become self.

It runs before __init__.

Why __new__ is needed for immutable objects:

Immutable objects like:

tuple
str
int

cannot be modified after creation, so customization must happen during object creation in __new__.

QUESTION 4
What __del__ does

Answer:

__del__ is called when an object is about to be destroyed.

Example:

```
class A:

    def __del__(self):
        print("object destroyed")
```

Problems with __del__:

1. Execution time is unpredictable
2. Garbage collector may delay it
3. Circular references may prevent execution

Better alternatives:

context managers
weakref.finalize()

QUESTION 5
Role of __dict__

Answer:

__dict__ is a dictionary storing object attributes.

Example:

```
class A:
    x = 10

a = A()
a.y = 20

a.__dict__
```

Output:

```
{'y': 20}
```

Class dictionary:

```
A.__dict__
```

contains:

methods
class attributes
descriptors

Problem:

Dictionaries consume memory.

Solution:

__slots__

Example:

```
class A:
    __slots__ = ('x', 'y')
```

Slots remove __dict__ and store attributes in a fixed memory structure, reducing memory usage.

QUESTION 6
Difference between __str__ and __repr__

Answer:

__str__

Human readable representation.

Used by:

print(obj)

__repr__

Unambiguous representation intended for developers.

Used by:

interactive interpreter
debugging

Example:

```
class A:

    def __repr__(self):
        return "A(10)"

    def __str__(self):
        return "value = 10"
```

QUESTION 7
Equality in Python objects

Answer:

a == b calls:

__eq__(self, other)

If __eq__ is overridden but __hash__ is not, Python disables hashing.

This prevents breaking the hash contract.

Python automatically sets:

__hash__ = None

Objects become unhashable.

QUESTION 8
How len(obj) works

Answer:

len(obj) calls:

obj.__len__()

Requirements:

Must return an integer >= 0.

Returning negative values raises:

ValueError

QUESTION 9
Iterator protocol

Answer:

Iterator protocol defines how objects are iterated.

Two required methods:

__iter__()
__next__()

Example:

```
class Counter:

    def __iter__(self):
        return self

    def __next__(self):
        raise StopIteration
```

iter(obj)

calls:

obj.__iter__()

next(obj)

calls:

obj.__next__()

Difference:

Iterable:
Object that can produce iterator.

Iterator:

Object that produces next values.

QUESTION 10
Purpose of __call__

Answer:

__call__ allows an object to behave like a function.

Example:

```
class Adder:

    def __init__(self, x):
        self.x = x

    def __call__(self, y):
        return self.x + y

a = Adder(10)

a(5)
```

Result:

15

Real world uses:

machine learning models
decorators
function objects

QUESTION 11
Context managers

Answer:

Context managers manage resource acquisition and release.

Used with:

with statement

Example:

with open("file.txt") as f:

Methods used:

__enter__()
__exit__()

Example:

```
class Manager:

    def __enter__(self):
        print("start")

    def __exit__(self, exc_type, exc, tb):
        print("cleanup")
```

If an exception occurs inside the block, __exit__ still runs.

QUESTION 12
Difference between __getattr__ and __getattribute__

Answer:

__getattribute__

Called for EVERY attribute access.

Example:

obj.attr

always triggers __getattribute__.

__getattr__

Called only if attribute is NOT found normally.

Flow:

1. __getattribute__
2. normal lookup
3. __getattr__ (fallback)

Overriding __getattribute__ is dangerous because it intercepts all attribute access.

QUESTION 13
Method Resolution Order (MRO)

Answer:

MRO defines the order in which base classes are searched when resolving methods.

Example:

class D(B, C)

Python determines which method to call using MRO.

QUESTION 14
C3 Linearization

Answer:

Python uses C3 linearization to compute MRO.

It guarantees:

1. consistent order
2. monotonic inheritance
3. predictable resolution

Example:

class D(B, C)

MRO:

D → B → C → A → object

QUESTION 15
How super() works

Answer:

super() does NOT mean "parent class".

It means:

next class in the MRO.

Example:

super().method()

calls the next implementation of method in the MRO chain.

This is critical for multiple inheritance.

QUESTION 16
Example resolution

```
class A:
    def f(self): print("A")

class B(A):
    def f(self): print("B")

class C(A):
    def f(self): print("C")

class D(B, C):
    pass

D().f()
```

Answer:

Output:

B

Because MRO is:

D → B → C → A → object

QUESTION 17
What is a descriptor

Answer:

A descriptor is any object implementing:

__get__()
__set__()
__delete__()

Descriptors customize attribute access.

Types:

Data descriptor:
Implements __set__ or __delete__

Non-data descriptor:
Only implements __get__

QUESTION 18
How property works

Answer:

property is implemented using descriptors.

Example:

class A:

```
    def __init__(self):
        self._x = 10

    @property
    def x(self):
        return self._x
```

Accessing:

```
a.x
```

internally calls:

```
property.__get__()
```

QUESTION 19
How methods are descriptors

Answer:

Functions inside classes are non-data descriptors.

When accessed through instance:

```
a.method
```

Python converts the function into a bound method.

Equivalent to:

```
A.method(a)
```

QUESTION 20
What is a metaclass

Answer:

A metaclass defines how classes are created.

Classes themselves are objects.

The default metaclass is:

type

Example:

```
class A:
    pass
```

is internally equivalent to:

```
A = type("A", (), {})
```

QUESTION 23
Abstract Base Classes

Answer:

Abstract Base Classes define interfaces that subclasses must implement.

Provided by:

abc module

QUESTION 24
@abstractmethod

Answer:

Marks methods that must be implemented by subclasses.

Example:

```
from abc import ABC, abstractmethod

class Shape(ABC):

    @abstractmethod
    def area(self):
```

```
    pass
```

If subclass does not implement it, instantiation raises:

TypeError

QUESTION 27
What Pydantic does

Answer:

Pydantic provides data validation and parsing using Python type hints.

Example:

```
class User(BaseModel):

    id: int
    name: str
```

It automatically:

- validates types
- converts input data
- parses JSON

Used heavily in APIs like FastAPI.

Tradeoff:

Validation overhead makes it slower than plain dataclasses.

PHASE 3 — FUNCTIONAL AND ADVANCED FEATURES
INTERVIEW REVIEW + STRONG ANSWERS
==========

QUESTION 1
What does it mean that functions are first class objects in Python?

Answer:

Functions in Python are first class objects, meaning they behave like any other object.

Because of this, functions can:

• be assigned to variables
• be passed as arguments to other functions
• be returned from functions
• be stored in data structures like lists or dictionaries
• be created dynamically

Example:

```
def greet():
    return "hello"

f = greet
print(f())
```

QUESTION 2
Explain this code.

```
def outer():
    def inner():
        return "hello"
    return inner

f = outer()
print(f())
```

Answer:

The function outer returns the function inner.

When outer() is called, it returns the function object inner, not the result of calling it.

So:

f = outer()

means f now refers to the function inner.

Calling:

f()

executes inner and prints "hello".

This demonstrates that functions are first class objects and can be returned from other functions.

QUESTION 3
What is a lambda function?

Answer:

A lambda function is an anonymous function defined using the lambda keyword.

Example:

lambda x: x * 2

Characteristics:

• no function name
• can contain only expressions
• returns the expression result automatically
• commonly used for short inline functions

Limitation:

Lambda cannot contain statements like assignment, loops, or multiple statements.

QUESTION 4
Explain:

sorted(data, key=lambda x: x[1])

Answer:

The key parameter specifies a function used to extract a comparison key from each element.

Here:

lambda x: x[1]

means:

take each element x and use the value at index 1 for sorting.

Example:

data = [(1,5),(2,3),(3,1)]

Sorting by second element produces:

[(3,1),(2,3),(1,5)]

QUESTION 5
How map() works

Answer:

map() applies a function to every element of an iterable.

Example:

map(square, [1,2,3])

In Python 3, map returns an iterator instead of a list.

Example:

result = map(lambda x: x*2, [1,2,3])

To get the values:

list(result)

Difference from list comprehension:

List comprehension is often more readable and flexible.

QUESTION 6
filter()

Answer:

filter() selects elements from an iterable for which a function returns True.

Example:

filter(lambda x: x%2==0, [1,2,3,4])

Result:

[2,4]

Many developers prefer list comprehensions:

[x for x in data if condition]

QUESTION 7
reduce()

Answer:

reduce() applies a function cumulatively to elements of an iterable.

Located in:

functools.reduce

Example:

reduce(lambda x,y: x+y, [1,2,3,4])

Steps:

1 + 2 → 3
3 + 3 → 6
6 + 4 → 10

Result:

10

Useful when reducing a sequence to a single value.

QUESTION 8
functools.lru_cache

Answer:

lru_cache caches results of function calls to avoid recomputation.

LRU means:

Least Recently Used.

Mechanism:

• results stored in dictionary
• key based on function arguments
• when cache size exceeds limit, least recently used entry is removed

Default size:

128

Implementation uses:

hash table + doubly linked list.

QUESTION 9
Difference between cache and lru_cache

Answer:

functools.cache

• unlimited cache size
• simple dictionary lookup
• no eviction

functools.lru_cache

• limited cache size
• removes least recently used items
• prevents unlimited memory growth

QUESTION 10
functools.partial

Example:

```
from functools import partial

def power(base, exp):
    return base ** exp

square = partial(power, exp=2)
```

Answer:

partial creates a new function with some arguments pre-filled.

square(5)

becomes

power(5,2)

Internally partial returns a callable object that stores:

• original function
• pre-filled arguments
• keyword arguments

QUESTION 11
functools.wraps

Answer:

wraps preserves metadata of the original function when creating decorators.

Without wraps:

function name
docstring
annotations

are lost.

Example:

```
from functools import wraps

def decorator(func):

    @wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)

    return wrapper
```

QUESTION 12
reduce deep explanation

Example:

```
reduce(lambda x,y: x+y, data)
```

Execution:

Take first two elements → apply function

Result becomes accumulator.

Then next element is combined with accumulator until iterable ends.

Optional third parameter can provide initial value.

QUESTION 13
What is a decorator

Answer:

A decorator is a function that modifies or extends the behavior of another function.

Decorators wrap a function and return a new function.

Example:

```
def decorator(func):

    def wrapper():
        print("before")
        func()
        print("after")

    return wrapper
```

QUESTION 14
What Python executes for decorators

Example:

```
@decorator
def func():
    pass
```

Python converts it internally to:

```
func = decorator(func)
```

QUESTION 16
Parameterized decorator

Example:

```
@retry(times=3)
```

Answer:

Parameterized decorators require an extra layer.

Structure:

decorator arguments → decorator factory → wrapper

Example:

```
def retry(times):

    def decorator(func):

        def wrapper(*args, **kwargs):
            for _ in range(times):
                return func(*args, **kwargs)

        return wrapper

    return decorator
```

QUESTION 18
Iterable vs Iterator vs Generator

Answer:

Iterable:

Object that can produce an iterator.

Examples:

list
tuple
set
dict

Iterator:

Object implementing:

__iter__()
__next__()

Generator:

Special iterator created using yield.

QUESTION 19
Generators

Example:

```
def gen():
    yield 1
    yield 2
```

Answer:

Generators produce values lazily.

When yield executes:

• function pauses
• state is saved
• execution resumes from same point on next call.

QUESTION 20
yield from

Answer:

yield from allows a generator to delegate iteration to another generator.

Example:

```
def gen():
    yield from range(3)
```

Equivalent to:

```
for x in range(3):
    yield x
```

It simplifies generator composition.

QUESTION 21
Generator methods

send(value)

Sends value into generator.

throw(exception)

Raises exception inside generator.

close()

Stops generator execution.

QUESTION 22
Generator internals

Answer:

Generators return a generator object.

This object stores:

• execution frame
• instruction pointer
• local variables
• stack state

When next() is called, execution resumes from the last yield.

QUESTION 23
with statement

Answer:

with statement simplifies resource management.

Example:

with open("file.txt") as f:

Python automatically closes the resource after block execution.

QUESTION 24
Internal execution of with

Python translates:

with open("file.txt") as f:

into:

```
mgr = open("file.txt")
val = mgr.__enter__()

try:
    f = val
    block
finally:
    mgr.__exit__()
```

QUESTION 25
Custom context manager

Example:

```
class Manager:

    def __enter__(self):
        print("enter")

    def __exit__(self, exc_type, exc, tb):
        print("exit")
```

QUESTION 26
contextlib.contextmanager

Answer:

contextmanager allows writing context managers using generators.

Example:

```
from contextlib import contextmanager

@contextmanager
def manager():

    print("enter")
    yield
    print("exit")
```

QUESTION 27
Async context managers

Example:

async with session.get(url):

Answer:

Async context managers use:

__aenter__()
__aexit__()

They are designed for asynchronous operations such as network requests.

PHASE 4 — IMPORT SYSTEM INTERNALS & PACKAGES
INTERVIEW REVIEW + STRONG ANSWERS
==========

QUESTION 1
What happens internally when Python executes:

import math

Answer:

Python performs these steps:

1. Check sys.modules
   Python first checks if the module already exists in sys.modules.

2. If present
   The cached module object is returned immediately.

3. If not present
   Python searches for the module using the import system.

4. Module loading
   Python locates the module using sys.meta_path and sys.path.

5. Create module object
   Python creates a new module object.

6. Execute module code
   The module source file is executed.

7. Cache module
   The module is stored in sys.modules for future imports.

QUESTION 2
What is sys.modules

Answer:

sys.modules is a dictionary that maps module names to module objects.

Example:

sys.modules["math"] → <module 'math'>

Purpose:

• Prevent re-importing modules
• Improve performance
• Handle circular imports

Deleting an entry:

del sys.modules["math"]

Forces Python to reload the module on the next import.

QUESTION 3
Import caching

Answer:

Python imports a module only once during program execution.

After the first import, the module object is stored in sys.modules.

Subsequent imports reuse the cached module.

Example:

```
import math
import math
```

The module code executes only once.

QUESTION 4
Circular imports

Example:

file A

```
import B
```

file B

```
import A
```

Answer:

Circular imports occur when two modules depend on each other during import.

Problem:

During import, Python places a partially initialized module in sys.modules.

If another module tries to access attributes that are not yet defined, an error occurs.

Common error:

ImportError or AttributeError.

Solutions:

• move imports inside functions
• refactor shared logic into a third module
• avoid circular dependencies

QUESTION 5
Module object

Answer:

A module is an object created by Python during import.

Modules contain attributes such as:

__name__
Module name.

__file__
Path of module file.

__package__
Package name.

__loader__
Loader used to load module.

__spec__
Module specification.

__dict__
Namespace containing module variables and functions.

QUESTION 6
Module spec

Answer:

Module specification (__spec__) describes how a module should be loaded.

It was introduced in PEP 451.

It contains metadata about the module such as:

• module name
• loader
• origin of module
• package information

The importlib system uses the module spec to load modules.

QUESTION 7
Python import search path

Answer:

Python searches modules using sys.path.

sys.path contains:

1. Current script directory
2. PYTHONPATH environment variable
3. Standard library directories
4. site-packages directory

Example:

```
import sys
print(sys.path)
```

If two modules have the same name, Python imports the first one found in sys.path.

QUESTION 8
Import hooks

Answer:

Import hooks allow customization of Python's import mechanism.

They are implemented using:

sys.meta_path

Use cases:

• loading modules from zip files
• loading modules from databases
• plugin systems
• sandboxed environments

SECTION 2 — PACKAGES

QUESTION 9
Difference between module and package

Answer:

Module:

A single Python file.

Example:

math.py

Package:

A directory containing multiple modules.

Example:

```
numpy/
   __init__.py
   core.py
   utils.py
```

Packages organize modules into hierarchical structures.

QUESTION 10
Role of __init__.py

Answer:

__init__.py marks a directory as a Python package.

It is executed when the package is imported.

Example:

import mypackage

mypackage/__init__.py runs.

Uses:

• initialize package
• expose package API
• import submodules automatically

Example:

```
# __init__.py
from .core import main_function
```

Note:

Python 3.3 introduced namespace packages where __init__.py is optional.

QUESTION 11
Absolute imports

Example:

from project.module import func

Answer:

Absolute imports specify the full path from the project root.

Advantages:

• clearer structure
• easier to understand
• recommended for large projects.

QUESTION 12
Relative imports

Example:

```
from .utils import helper
from ..core import service
```

Answer:

Relative imports refer to modules relative to the current package.

Symbols:

. → current package
.. → parent package

Example structure:

```
project/
  core/
  utils/
```

Relative imports are useful for internal package structure.

QUESTION 13
Why this fails

python module.py

but works with

python -m module

Answer:

When running a script directly:

python module.py

Python treats the file as a top-level script.

__package__ is None.

Relative imports fail because Python does not know the package context.

When using:

python -m module

Python runs the module within its package context.

This allows relative imports to work correctly.

QUESTION 14
Namespace packages

Answer:

Namespace packages allow a package to be split across multiple directories.

They do not require __init__.py.

Example:

```
package/
  part1/
  part2/
```

Python combines them into a single logical package.

Use cases:

• large distributed libraries
• plugin architectures

QUESTION 15
How Python resolves this import

from package.subpackage.module import func

Steps:

1. Locate "package" using sys.path.
2. Load package and execute package/__init__.py.
3. Locate subpackage inside package.
4. Execute subpackage/__init__.py.
5. Locate module file.
6. Create module object.
7. Execute module code.
8. Store module in sys.modules.
9. Retrieve func from module namespace.

PHASE 5 — CONCURRENCY AND PARALLELISM
DEEP INTERVIEW Q&A (GRILL LEVEL)
==========

SECTION 1 — THREADING

Q1. What is a thread?

Answer:

A thread is the smallest unit of execution within a process. Multiple threads share the same memory space of a process but execute independently.

In Python, threads are managed using the threading module.

Example:

```
import threading

def task():
    print("running")

t = threading.Thread(target=task)
t.start()
```

Key property:
Threads share memory, which allows fast communication but requires synchronization mechanisms.

Q2. Why does Python have the GIL and how does it affect threading?

Answer:

The Global Interpreter Lock (GIL) ensures that only one thread executes Python bytecode at a time in CPython.

Purpose:

• protects internal interpreter state
• simplifies memory management
• prevents race conditions in reference counting

Impact:

CPU-bound tasks do NOT benefit from threading because threads compete for the GIL.

However IO-bound tasks benefit because the GIL is released during blocking operations.

Q3. What is a race condition?

Answer:

A race condition occurs when multiple threads access shared data and the final result depends on execution order.

Example:

counter += 1

This operation is not atomic and can cause inconsistent results when multiple threads modify it simultaneously.

Q4. What is a Lock?

Answer:

A lock ensures that only one thread can access a critical section at a time.

Example:

```
import threading

lock = threading.Lock()

with lock:
    critical_section()
```

When a thread acquires the lock, other threads attempting to acquire it must wait.

Q5. Difference between Lock and RLock.

Answer:

Lock
• basic mutual exclusion
• cannot be acquired twice by same thread

RLock (reentrant lock)
• same thread can acquire lock multiple times
• must release it same number of times

Useful in recursive functions or nested locking scenarios.

Q6. What is a Condition variable?

Answer:

A Condition allows threads to wait until a certain condition is met.

Example:

condition = threading.Condition()

Used for:

• producer-consumer problems
• coordination between threads

Key methods:

wait()
notify()
notify_all()

Q7. What is an Event?

Answer:

An Event is a synchronization primitive used for signaling between threads.

Example:

event = threading.Event()

Thread waits:

event.wait()

Another thread signals:

event.set()

Use case:

Thread waits until some event occurs.

Q8. What is a Semaphore?

Answer:

A semaphore controls access to a limited number of resources.

Example:

sem = threading.Semaphore(3)

At most 3 threads can enter the critical section simultaneously.

Useful for:

connection pools
rate limiting

==========

SECTION 2 — concurrent.futures

Q9. What is concurrent.futures?

Answer:

concurrent.futures is a high-level interface for asynchronous execution of tasks.

It simplifies thread and process pools.

Executors:

ThreadPoolExecutor
ProcessPoolExecutor

Q10. ThreadPoolExecutor example

Example:

```
from concurrent.futures import ThreadPoolExecutor

def work(x):
    return x*x

with ThreadPoolExecutor(max_workers=4) as executor:
    results = executor.map(work, [1,2,3])
```

Q11. submit() vs map()

Answer:

submit()

• schedules a single task
• returns a Future object
• supports different arguments per call

Example:

```
future = executor.submit(func, arg)
```

map()

• applies function to iterable
• returns results in order

• simpler API

Q12. What is a Future?

Answer:

A Future represents the result of a computation that may complete in the future.

Key methods:

future.result()
future.done()
future.exception()

Q13. What is as_completed?

Answer:

as_completed yields futures as soon as they finish.

Example:

```
from concurrent.futures import as_completed

for future in as_completed(futures):
    print(future.result())
```

Advantage:

Process results immediately instead of waiting for all tasks.

Q14. Exception handling in futures

Answer:

Exceptions raised inside worker threads are captured by the Future.

Example:

```
try:
    result = future.result()
except Exception as e:
    print("error", e)
```

Q15. Why prefer concurrent.futures over threading module?

Answer:

Advantages:

• simpler API
• automatic thread management
• better error handling
• easier scaling

It abstracts thread pool management.

==========

SECTION 3 — MULTIPROCESSING

Q16. Why multiprocessing exists in Python?

Answer:

Multiprocessing bypasses the GIL by using separate processes instead of threads.

Each process has its own Python interpreter and memory space.

This allows true parallel execution on multiple CPU cores.

Q17. fork vs spawn

Answer:

fork

• copies parent process memory
• faster startup

• available on Unix systems

spawn

• starts fresh Python interpreter
• imports module again
• default on Windows

Q18. Inter Process Communication (IPC)

Answer:

Processes do not share memory, so communication happens through IPC.

Common IPC mechanisms:

multiprocessing.Queue
multiprocessing.Pipe
shared memory
sockets

Q19. Shared memory

Answer:

Shared memory allows processes to access common memory.

Example:

multiprocessing.Value
multiprocessing.Array

Python 3.8+ provides:

multiprocessing.shared_memory

Used for high performance data sharing.

==========

SECTION 4 — ASYNCIO

Q20. What is asyncio?

Answer:

Asyncio is Python's framework for asynchronous programming using a single thread and event loop.

It is designed for IO-bound tasks like:

• network requests
• database queries
• file operations

Q21. What is an event loop?

Answer:

The event loop schedules and runs asynchronous tasks.

Responsibilities:

• manage tasks
• wait for IO events
• resume coroutines when ready

Q22. What is a coroutine?

Answer:

A coroutine is a function defined with async def.

Example:

```
async def fetch():
    await network_call()
```

Coroutines can pause execution using await.

Q23. What is a Task?

Answer:

A Task is a wrapper around a coroutine that schedules it on the event loop.

Example:

task = asyncio.create_task(coro())

Q24. What is a Future in asyncio?

Answer:

A Future represents a result that will become available later.

Tasks internally use Futures to store results.

Q25. What does asyncio.gather do?

Answer:

gather runs multiple coroutines concurrently and waits for all of them.

Example:

results = await asyncio.gather(coro1(), coro2())

Q26. Cancellation

Answer:

Tasks can be cancelled.

Example:

task.cancel()

The coroutine receives a CancelledError.

Q27. Async context managers

Answer:

Async context managers use:

__aenter__()
__aexit__()

Example:

```
async with session.get(url) as resp:
    data = await resp.text()
```

==========

SECTION 5 — CHOOSING THE RIGHT MODEL

Q28. CPU bound vs IO bound

Answer:

CPU-bound tasks

• heavy computations
• use multiprocessing

IO-bound tasks

• waiting on network or disk
• use threading or asyncio

Q29. When to choose threading?

Answer:

Use threading when:

• performing IO-bound work
• interacting with blocking libraries

Q30. When to choose asyncio?

Answer:

Use asyncio when:

• performing many concurrent IO operations
• working with async-compatible libraries

Q31. When to choose multiprocessing?

Answer:

Use multiprocessing when:

• performing CPU-heavy work
• needing true parallelism

Q32. Scaling strategies in Python

Answer:

Typical production strategy:

IO heavy systems:
asyncio + event loop

CPU heavy workloads:
multiprocessing

Mixed workloads:
thread pools + async services

Distributed systems:
message queues + worker processes
PHASE 6 — MEMORY AND PERFORMANCE

DEEP INTERVIEW Q&A (GRILL LEVEL)
==========

SECTION 1 — MEMORY MANAGEMENT

Q1. How does Python manage memory?

Answer:

Python primarily uses **reference counting** for memory management.

Every object keeps track of how many references point to it.

Example:

a = []
b = a

The list object now has reference count = 2.

When references drop to zero, Python immediately frees the object.

Additionally, Python uses a **cyclic garbage collector** to handle reference cycles that reference counting alone cannot resolve.

Q2. What is reference counting?

Answer:

Reference counting tracks the number of references to an object.

Example:

a = [1,2,3]

Reference count increases when:

• assigning object to variable
• passing object as function argument
• storing object in container

Reference count decreases when:

• variable goes out of scope
• variable reassigned
• object removed from container

When reference count becomes zero:

The object is immediately destroyed.

Q3. Why does Python need a garbage collector if reference counting already exists?

Answer:

Reference counting fails with circular references.

Example:

```
class A:
    pass

a = A()
b = A()

a.ref = b
b.ref = a
```

Even if both objects are unused, their reference count never becomes zero.

Python's cyclic garbage collector detects such cycles and frees them.

Q4. Explain Python's garbage collector generations.

Answer:

Python uses a **generational garbage collector**.

Objects are divided into three generations:

Generation 0
New objects. Collected frequently.

Generation 1
Objects that survived Gen 0 collection.

Generation 2
Long-lived objects. Collected rarely.

Idea:

Most objects die young, so frequent checks in Gen 0 reduce overhead.

Q5. What does the gc module do?

Answer:

The gc module provides access to Python's garbage collector.

Useful functions:

gc.collect()
Forces garbage collection.

gc.get_count()
Returns number of objects in each generation.

gc.get_objects()
Lists objects tracked by the GC.

gc.disable()
Disables cyclic garbage collector.

Q6. When would you manually call gc.collect()?

Answer:

Rarely needed, but useful when:

• large temporary object graphs exist
• long-running processes
• memory-sensitive applications

• testing memory leaks

==========

SECTION 2 — PERFORMANCE PROFILING

Q7. What is profiling?

Answer:

Profiling measures where a program spends time or memory.

It helps identify performance bottlenecks.

Q8. What is cProfile?

Answer:

cProfile is Python's built-in deterministic profiler.

It records:

• function call count
• total execution time
• cumulative time

Example:

```
import cProfile

cProfile.run("my_function()")
```

Q9. What is timeit?

Answer:

timeit measures execution time of small code snippets.

Example:

```
import timeit

timeit.timeit("x*x", number=1000000)
```

It disables garbage collection and runs multiple iterations to reduce noise.

Q10. What is memory_profiler?

Answer:

memory_profiler tracks memory usage of Python code line by line.

Example:

```
from memory_profiler import profile

@profile
def func():
    x = [i for i in range(1000000)]
```

Q11. What is tracemalloc?

Answer:

tracemalloc tracks memory allocations in Python.

Example:

```
import tracemalloc

tracemalloc.start()
```

It allows developers to see where memory is allocated.

==========

SECTION 3 — BIG O OF BUILTINS

Q12. Time complexity of Python list operations

Answer:

Access by index

O(1)

Append

O(1) amortized

Insert at beginning

O(n)

Remove element

O(n)

Iteration

O(n)

Q13. Time complexity of Python dictionary operations

Answer:

Lookup

O(1) average

Insertion

O(1) average

Deletion

O(1) average

Worst case

O(n) if many hash collisions occur.

Q14. Why dictionary operations are O(1)?

Answer:

Dictionaries use a hash table.

Keys are hashed to determine index.

Collision resolution uses probing.

Average case remains constant time.

==========

SECTION 4 — OPTIMIZATIONS

Q15. What is __slots__?

Answer:

__slots__ reduces memory usage by removing instance dictionaries.

Example:

```
class A:
    __slots__ = ('x','y')
```

Benefits:

• lower memory footprint
• faster attribute access

Limitations:

• no dynamic attribute creation
• inheritance complexities

Q16. Why generators improve performance?

Answer:

Generators produce values lazily instead of creating entire collections.

Example:

sum(x*x for x in range(10**9))

Advantages:

• lower memory usage
• faster processing of large datasets

Q17. What are temporary allocations and why avoid them?

Answer:

Temporary objects increase memory pressure and CPU overhead.

Example:

Bad:

result = sum([x*x for x in data])

Better:

result = sum(x*x for x in data)

The generator avoids creating an intermediate list.

Q18. What are C extensions?

Answer:

C extensions allow Python modules to be written in C for better performance.

Examples:

NumPy
Pandas

Advantages:

• direct memory control
• faster execution
• bypass Python interpreter overhead

Q19. What is Cython?

Answer:

Cython is a superset of Python that compiles to C.

Example:

```
def square(int x):
    return x*x
```

Benefits:

• static typing
• faster execution
• easy integration with C libraries

==========

SECTION 5 — PERFORMANCE INTERVIEW QUESTIONS

Q20. Why Python is slower than C?

Answer:

Python is slower because:

• dynamic typing
• interpreter overhead
• garbage collection
• high-level abstractions

However performance-critical parts are often written in C.

Q21. Common Python performance optimization strategies

Answer:

• use built-in functions
• avoid unnecessary object creation
• use generators for large data
• use appropriate data structures
• move heavy computation to C extensions
• use multiprocessing for CPU tasks

Q22. How would you diagnose a Python memory leak?

Answer:

Typical workflow:

1. use tracemalloc
2. inspect object growth
3. use gc.get_objects()
4. analyze reference cycles
5. profile allocations

==========

END OF PHASE 6
PHASE 7–13 — PRODUCTION PYTHON, RUNTIME, NETWORKING, TESTING, PACKAGING, SECURITY
HIGH-VALUE INTERVIEW Q&A (NO LOW VALUE TOPICS)
==========

==========
PHASE 7 — PYTHON RUNTIME & SYSTEM INTERACTION
==========

Q1. What is the sys module and why is it important?

Answer:

The sys module provides access to Python runtime internals.

Commonly used attributes:

sys.argv
Command line arguments.

sys.path
List of directories Python searches for modules.

sys.modules
Cache of imported modules.

sys.version
Python interpreter version.

sys.getsizeof(obj)
Returns size of object in bytes.

Q2. What is sys.path and how does Python use it?

Answer:

sys.path is a list of directories Python searches when importing modules.

Search order typically:

1. Current script directory
2. PYTHONPATH environment variable
3. Standard library directories
4. site-packages

Example:

```
import sys
print(sys.path)
```

Q3. What is a Python stack frame?

Answer:

A stack frame represents the execution state of a function call.

It contains:

• local variables
• instruction pointer
• operand stack
• references to globals and builtins
• link to previous frame

Frames form the call stack.

Q4. What is the traceback module used for?

Answer:

traceback module helps inspect stack traces after exceptions.

Example:

```
import traceback

try:
    x = 1/0
except:
    traceback.print_exc()
```

Used for debugging and logging.

Q5. How do you run external commands safely in Python?

Answer:

Use subprocess module.

Example:

```
import subprocess

result = subprocess.run(
    ["ls","-l"],
    capture_output=True,
    text=True
)

print(result.stdout)
```

Never pass user input directly to shell commands to avoid injection.

Q6. Difference between subprocess.run and Popen.

Answer:

subprocess.run
High level interface that runs command and waits for completion.

Popen
Lower level interface for streaming interaction with processes.

Example:

```
process = subprocess.Popen(
    ["python","script.py"],
    stdout=subprocess.PIPE
)
```

Q7. What are environment variables in Python?

Answer:

Environment variables are accessed using os.environ.

Example:

```
import os

db_url = os.environ.get("DATABASE_URL")
```

Used for configuration and secrets management.

==========
PHASE 8 — LOGGING AND CONFIGURATION
==========

Q8. Why should production systems use logging instead of print?

Answer:

print()

- not structured
- no log levels
- difficult to manage in production

logging module provides:

- log levels
- multiple outputs
- structured formatting

Q9. Basic logging example.

Answer:

```
import logging

logging.basicConfig(level=logging.INFO)

logging.info("server started")
logging.error("error occurred")
```

Q10. What are logging levels?

Answer:

DEBUG
INFO
WARNING
ERROR
CRITICAL

Used to categorize severity of events.

Q11. What are handlers in logging?

Answer:

Handlers determine where logs go.

Examples:

StreamHandler
FileHandler
SocketHandler
HTTPHandler

==========
PHASE 9 — NETWORKING & SERIALIZATION
==========

Q12. Why is pickle dangerous?

Answer:

pickle can execute arbitrary code during deserialization.

Loading untrusted pickle data can lead to remote code execution.

Therefore never unpickle data from untrusted sources.

Q13. Difference between JSON and pickle.

Answer:

JSON

• human readable
• language independent
• safe

pickle

• Python specific
• supports complex objects
• unsafe if untrusted

Q14. What is connection pooling in HTTP clients?

Answer:

Connection pooling reuses TCP connections instead of opening new ones for every request.

Benefits:

• faster requests
• lower latency
• fewer system resources

Q15. Why should HTTP requests always use timeouts?

Answer:

Without timeouts requests may hang indefinitely.

Example:

requests.get(url, timeout=5)

==========
PHASE 10 — TESTING & RELIABILITY
==========

Q16. Difference between unittest and pytest.

Answer:

unittest

• built-in library
• class-based tests

pytest

• simpler syntax
• fixtures
• powerful plugin ecosystem

Q17. What is mocking?

Answer:

Mocking replaces real dependencies with fake objects during testing.

Example:

from unittest.mock import patch

Q18. Why mocking is useful.

Answer:

• isolate units of code
• avoid external services

• make tests deterministic

Q19. What are fixtures in pytest?

Answer:

Fixtures provide reusable test setup.

Example:

```
@pytest.fixture
def db_connection():
    return create_db()
```

==========
PHASE 11 — PACKAGING & DISTRIBUTION
==========

Q20. What is pyproject.toml?

Answer:

pyproject.toml defines build system and project metadata.

It replaces setup.py in modern packaging.

Example fields:

```
[project]
name
version
dependencies
```

Q21. What is a wheel?

Answer:

A wheel (.whl) is a prebuilt Python package.

Benefits:

• faster installation
• no compilation required

Q22. What are entry points?

Answer:

Entry points allow Python packages to expose CLI commands.

Example:

pip install black

creates a command:

black file.py

==========
PHASE 12 — PERFORMANCE IN PRODUCTION
==========

Q23. How would you detect a Python memory leak?

Answer:

Tools:

tracemalloc
gc module
memory_profiler

Steps:

1. monitor memory growth
2. inspect object references
3. identify cycles

Q24. What are caching strategies in Python?

Answer:

Common caches:

in-memory LRU cache
Redis cache
disk cache

Example:

functools.lru_cache

Q25. Why caching improves performance?

Answer:

Caching avoids recomputing expensive operations.

Example:

database queries
API responses
complex computations

==========
PHASE 13 — PYTHON SECURITY
==========

Q26. What is command injection in Python?

Answer:

Occurs when user input is directly passed to shell commands.

Example (dangerous):

subprocess.run(f"rm {filename}", shell=True)

Safe version:

subprocess.run(["rm", filename])

Q27. What is dependency vulnerability?

Answer:

Using outdated libraries may expose security issues.

Tools to detect:

pip-audit
safety

Q28. Why should secrets not be stored in source code?

Answer:

Hardcoded secrets may leak through:

• Git repositories
• logs
• backups

Better practice:

environment variables
secret managers
vault services

==========
END OF ADVANCED PYTHON PRODUCTION TOPICS
==========

FastAPI

PHASE 0: WEB AND HTTP FUNDAMENTALS (FOUNDATION)

Before touching FastAPI you must understand the **web protocol layer**.

1. HTTP Protocol Deep Dive
    - Request / Response structure
    - Headers
    - Status codes
    - Content negotiation
    - Cookies vs Headers
2. HTTP Methods Semantics
    - GET
    - POST
    - PUT
    - PATCH
    - DELETE
    - Idempotency
3. REST Principles
    - Resource modeling
    - Statelessness
    - Hypermedia
    - Versioning
4. Serialization

    - JSON encoding
    - MIME types
    - Content-Type vs Accept
5. API Design Basics
    - Pagination
    - Filtering
    - Sorting
    - Error formats

Without this phase FastAPI feels like magic.

PHASE 1: ASGI AND THE MODERN PYTHON WEB STACK

FastAPI is built on **ASGI**, so understand the runtime.

1. WSGI vs ASGI
    - Blocking vs async servers
    - Request lifecycle difference
2. ASGI Specification
    - Scope
    - Receive
    - Send
    - Lifespan events
3. ASGI Servers

    - Uvicorn
    - Hypercorn
    - Daphne
4. Event Loop Interaction
    - asyncio event loop
    - concurrency model for web servers
5. Request Lifecycle in ASGI

Client → Server → Middleware → App → Response

PHASE 2: FASTAPI CORE ARCHITECTURE

Now the FastAPI layer makes sense.

1. FastAPI Application Object

app = FastAPI()

What it actually does internally.

2. Path Operations

@app.get()
@app.post()

Concepts:

- routing
- operation id
- path matching

3. Path Parameters

/users/{id}

- type conversion
- validation

4. Query Parameters

/items?limit=10

5. Request Body Handling

FastAPI automatically parses JSON.

6. Response Models

- response_model
- serialization control

PHASE 3: PYDANTIC (THE DATA LAYER)

FastAPI heavily depends on Pydantic.

1. Pydantic BaseModel

```
class User(BaseModel):
  name: str
  age: int
```

2. Validation Pipeline

JSON → parse → validate → Python object.

3. Field Validation

- Field()
- validators

- constraints

4. Nested Models

User -> Address

5. Model Serialization

model.dict()
model.json()

6. ORM Mode

from_attributes = True

7. Custom Validators

PHASE 4: REQUEST AND RESPONSE LIFECYCLE

Understanding how data flows through FastAPI.

1. Request Object

from fastapi import Request

2. Response Object

- JSONResponse
- PlainTextResponse
- StreamingResponse
- FileResponse

3. Headers

4. Cookies

5. Background Tasks

PHASE 5: DEPENDENCY INJECTION SYSTEM (THE CORE SUPERPOWER)

This is the **most important FastAPI concept**.

1. Depends()

```
def get_db():
  yield db
```

2. Dependency Graph

FastAPI builds a dependency tree automatically.

3. Scoped Dependencies

- request scoped
- app scoped

4. Sub-dependencies

auth -> db -> config

5. Dependency caching
6. Yield dependencies (context manager style)

PHASE 6: APPLICATION STRUCTURE AND ARCHITECTURE

Now we design **production systems**.

Typical architecture:

app/
  api/
  services/
  models/
  schemas/
  db/
  core/

Concepts:

1. Router Separation

APIRouter()

2. Modular APIs

3. Service Layer Pattern

Controllers → Services → Repository

4. Repository Pattern

Abstract DB operations.

5. Configuration Management

pydantic Settings

PHASE 7: DATABASE INTEGRATION

Most FastAPI apps exist to talk to a DB.

1. SQLAlchemy Core Concepts

- Engine
- Session
- ORM models

2. Session Lifecycle

Session per request pattern.

3. Async SQLAlchemy

async_sessionmaker

4. Migrations

Alembic.

5. Connection Pooling

6. Transaction Handling

PHASE 8: AUTHENTICATION AND SECURITY

Real production APIs require security.

1. OAuth2

2. JWT Tokens

3. Password Hashing

bcrypt / passlib.

4. Security Dependencies

OAuth2PasswordBearer

5. Role Based Access Control

RBAC.

6. API Keys

PHASE 9: MIDDLEWARE AND CROSS CUTTING CONCERNS

Middleware runs before the route handler.

1. Middleware Concept
2. Custom Middleware

@app.middleware("http")

3. CORS
4. Rate Limiting
5. Logging Middleware
6. Request ID Tracking

PHASE 10: PERFORMANCE AND SCALING

For large scale services.

1. Async vs Sync Endpoints
2. Threadpool Execution
3. Connection Pooling
4. Caching

Redis

5. Streaming responses
6. Background workers

Celery / Arq / Dramatiq.

PHASE 11: TESTING FASTAPI APPLICATIONS

1. TestClient
2. pytest integration
3. Dependency overrides
4. Database test isolation
5. API contract testing

PHASE 13: PRODUCTION DEPLOYMENT

1. Gunicorn + Uvicorn workers
2. Containerization

Docker.

3. Reverse proxy

Nginx.

4. Load balancing
5. Health checks

6. Graceful shutdown

PHASE 14: ADVANCED FASTAPI PATTERNS

This is senior level mastery.

1. Lifespan events

@asynccontextmanager

2. Custom APIRoute
3. Custom Request classes
4. Dependency injection containers
5. Multi-tenancy
6. API versioning
7. GraphQL with FastAPI

Q&A

HTTP Revision NotesQ1. HTTP Request Structure

An HTTP request has **four** main parts:

1. **Request Line** (Mandatory)
2. **Headers** (Mandatory: `Host` for HTTP/1.1)
3. **Blank Line** (Separator)
4. **Optional Body** (Payload)

Example (POST Request):
HTTP Request Structure

```
POST /users HTTP/1.1              <- Request Line
Host: api.example.com             <- Headers
Content-Type: application/json
Authorization: Bearer token123

                                  <- Blank Line
{                                 <- Body
  "name": "Vikas",
  "age": 25
                                              }
                                              }
```

Component	Content
Request Line	Method (`POST`), Resource Path (`/users`), HTTP Version (`HTTP/1.1`)
Headers	Key-value metadata (e.g., `Host`, `Authorization`, `Content-Type`)
Blank Line	Separates Headers from the Body.
Body	Optional payload (mainly for `POST`, `PUT`, `PATCH`). `GET` usually has no body.

Mandatory for HTTP/1.1:

1. Request Line
2. `Host` header

-----Q2. Request Analysis & Headers

Request Example:
GET /orders?id=10 HTTP/1.1
Host: api.shop.com
Authorization: Bearer abc123
Accept: application/json

Component	Value	Notes
Resource Identifier	`/orders`	The specific resource.
Query Parameters	`?id=10`	Key (`id`) and Value (`10`) used to filter or specify the resource.

Why is `Authorization` a Header?

- Headers carry **metadata** for request processing (like authentication).
- Credentials must be independent of the request **payload/body**.
- Many requests (`GET`) do not have a body.

Missing `Host` Header:

- HTTP/1.1 **requires** the `Host` header.
- Server will respond with **`400 Bad Request`**.

Reverse Proxy and `Host` Header:

- Proxies host multiple domains on one IP.
- The **`Host` header** tells the proxy **which upstream server** to route the request to (e.g., `api.shop.com` -> Backend A, `admin.shop.com` -> Backend B).

-----Q3. Full Lifecycle of an HTTP Request

1. **DNS Resolution:** Browser resolves the domain name (e.g., `api.github.com`) to an IP address.
2. **TCP Handshake (3-way):**
    - Client -> `SYN`
    - Server -> `SYN ACK`
    - Client -> `ACK`

    - *Result: TCP connection established.*
3. **TLS Handshake (HTTPS only):**
    - **ClientHello:** Sends supported TLS versions/cipher suites.
    - **ServerHello:** Chooses cipher, sends server certificate.
    - Client verifies the certificate using the CA.
    - Uses **ECDHE** to derive a shared session key for encryption.
4. **HTTP Request:** Client sends the HTTP request over the encrypted TLS channel.
5. **Server Processing:** Server processes the request (application logic, DB queries, etc.).
6. **HTTP Response:** Server sends the response:
   HTTP/1.1 200 OK
7. Content-Type: application/json
8. { data }
9. **Connection Reuse (Keep Alive):**
    - **HTTP/1.1:** Multiple requests can reuse the same TCP connection.
    - **HTTP/2:** Multiplexes many streams over a single connection.

-----Q4. Hop-by-Hop vs. End-to-End Headers

Header Type	Description	Proxy Behavior	Examples
Hop-by-Hop	Consumed by proxies/intermediaries; not forwarded to the final server.	**Removed** by proxies.	`Connection`, `Keep-Alive`, `Transfer-Encoding`, `Upgrade`, `Proxy-Authenticate`
End-to-End	Travel from the client to the final server unchanged.	**Forwarded** by proxies.	`Authorization`, `Content-Type`, `Accept`, `Cache-Control`, `User-Agent`

-----Q5. Content Negotiation

Uses the **`Accept`** header to allow the client to specify the preferred format for the **response**.

Example:

```
Accept: application/json, application/xml;q=0.9, */*;q=0.8
```

- `application/json` -> Preference weight **q=1.0** (Highest)
- `application/xml` -> **q=0.9**
- `*/*` (any) -> **q=0.8**

Server Logic:

- Chooses the **highest supported** format (e.g., if supports JSON/XML -> returns JSON).
- If server supports **none** of the formats -> responds with **`406 Not Acceptable`**.

Key Difference:

- **`Accept`**: Format the **Client expects** in the **Response**.
- **`Content-Type`**: Format of the **Request Body** being **Sent**.

-----Q6. Cookies vs. Headers

Why Cookies Exist:

- HTTP is **stateless**.
- Cookies allow servers to **maintain session state** on the client side.

Cookie Flow:

1. **Server sets:** Sends `Set-Cookie: session_id=abc123` in the **Response** header.
2. **Browser stores:** Saves it in the client-side cookie store.
3. **Browser sends:** Automatically includes `Cookie: session_id=abc123` in future **Request** headers.

Security Issues & Flags:

Issue	Flag	Purpose

Session hijacking, XSS	`HttpOnly`	JavaScript **cannot read** the cookie (prevents XSS theft).
MITM attacks	`Secure`	Cookie sent **only over HTTPS**.
CSRF attacks	`SameSite`	Restricts cross-site sending to prevent CSRF.

JWT vs. Cookies:

- **JWT (JSON Web Tokens)** allows **stateless authentication**. The server does not need to store session information, as the token is self-contained.

-----Q7. HTTP Method Semantics

Method	Purpose	Safe (Read-only)	Idempotent (Same result multiple times)
GET	Retrieve resource.	**Yes**	**Yes**
POST	Create **new** resource.	**No**	**No (Creates n…**
PUT	**Replace entire** resource.	**No**	**Yes (Results i…**
PATCH	**Partially update** resource.	**No**	Usually No
DELETE	Delete resource.	**No**	**Yes**

Why PUT is Idempotent: Sending the same `PUT` request multiple times (e.g., to replace `/users/1`) results in the *same final state* for that resource.-----Q9. Serialization and MIME Types

MIME types describe the format of the data being transmitted.

MIME Type	Description	Use Case

`application/json`	Structured JSON data.	APIs, general data exchange.
`application/xml`	XML formatted data.	Older systems, specific requirements.
`application/x-www-form-urlencoded`	HTML form submissions (key=value&key2=value2).	Simple form data.
`multipart/form-data`	Data sent in multiple parts.	**File uploads** (allows streaming binary content).

Why File Uploads Cannot Use JSON:

- Files are typically **binary data**.
- JSON supports only **text** data.
- `multipart/form-data` is necessary to stream binary file content.

PHASE 1: ASGI AND THE MODERN PYTHON WEB STACK — INTERVIEW Q&A

1. Q: What problem did ASGI solve that WSGI could not?
A: WSGI only supports synchronous request-response cycles and blocks during I/O. ASGI supports asynchronous communication, allowing concurrency, WebSockets, and long-lived connections using async event loops.

2. Q: What is the fundamental architectural difference between WSGI and ASGI?
A: WSGI handles requests synchronously using threads or processes, while ASGI handles requests asynchronously using coroutines and an event loop.

3. Q: Why is WSGI considered blocking?
A: In WSGI, a worker thread handles a request until completion. If it waits on I/O like DB or network, the thread is blocked and cannot serve other requests.

4. Q: Why is ASGI non-blocking?
A: ASGI uses asynchronous coroutines that yield control when waiting for I/O, allowing the event loop to schedule other requests during the wait.

5. Q: In WSGI, how is concurrency typically achieved?

A: By spawning multiple threads or processes using servers like Gunicorn or uWSGI.

6. Q: In ASGI, how is concurrency achieved?
A: Through an asyncio event loop that schedules multiple coroutines and uses non-blocking I/O.

7. Q: Write the basic ASGI application interface.
A: async def app(scope, receive, send):

8. Q: What are the three core arguments passed to an ASGI application?
A: scope, receive, send.

9. Q: What does the "scope" object represent in ASGI?
A: It contains metadata about the connection such as request method, path, headers, client address, and protocol type.

10. Q: Give examples of information stored in the ASGI scope.
A: HTTP method, path, headers, query string, client IP, server info, protocol type (http/websocket).

11. Q: What is the purpose of the receive function in ASGI?
A: It asynchronously receives messages from the client, such as request body chunks or websocket messages.

12. Q: What is the purpose of the send function in ASGI?
A: It sends messages back to the client, such as response headers and response body.

13. Q: What are the two main messages sent when returning an HTTP response in ASGI?
A: http.response.start and http.response.body.

14. Q: What is an example of a message received through receive()?
A: http.request or websocket.receive.

15. Q: What are ASGI lifespan events?
A: Events triggered during application startup and shutdown.

16. Q: What are the two standard lifespan events?
A: startup and shutdown.

17. Q: Why are lifespan events important in web applications?
A: They allow initialization and cleanup of resources like database pools, caches, or ML models.

18. Q: Name three popular ASGI servers.
A: Uvicorn, Hypercorn, Daphne.

19. Q: Which ASGI server is most commonly used with FastAPI?
A: Uvicorn.

20. Q: Why is Uvicorn considered fast?
A: It uses uvloop (a faster event loop) and httptools (fast HTTP parsing).

21. Q: Which ASGI server supports HTTP/2 and HTTP/3 more extensively?
A: Hypercorn.

22. Q: Which ASGI server is mainly used with Django Channels?
A: Daphne.

23. Q: What is an asyncio event loop?
A: A scheduler that manages and executes asynchronous tasks, handling I/O events and switching between coroutines.

24. Q: What happens when an async function encounters I/O in an event loop?
A: It yields control back to the event loop so other tasks can execute.

25. Q: Why can async servers handle thousands of concurrent connections?
A: Because they do not block threads during I/O and instead schedule coroutines efficiently on a single event loop.

26. Q: What type of workloads benefit most from async web servers?
A: I/O bound workloads such as database queries, API calls, or file operations.

27. Q: Why do async servers not significantly improve CPU-bound workloads?
A: Because CPU-bound tasks do not yield control and block the event loop.

28. Q: What is the typical request lifecycle in an ASGI application?
A: Client → ASGI server → scope creation → middleware → router → endpoint → response → send back to client.

29. Q: Where does middleware execute in the ASGI lifecycle?
A: Between the ASGI server and the application endpoint.

30. Q: What role does the ASGI server play in request processing?
A: It accepts connections, creates the ASGI scope, runs the event loop, and invokes the application callable.

31. Q: What happens after Uvicorn receives a request?
A: It creates the scope and invokes the application coroutine with scope, receive, and send.

32. Q: Why is ASGI necessary for WebSockets?
A: Because WebSockets require bidirectional asynchronous communication, which WSGI cannot support.

33. Q: What is the difference between concurrency and parallelism in async servers?
A: Concurrency handles many tasks by switching between them, while parallelism executes multiple tasks simultaneously using multiple CPU cores.

34. Q: How do ASGI servers achieve parallelism if needed?
A: By running multiple worker processes.

35. Q: Why might you still run multiple workers with Uvicorn?
A: To utilize multiple CPU cores and increase throughput.

SECTION 1 — WHY PYDANTIC EXISTS

Q1. What problem does Pydantic solve?

Answer:

Pydantic provides **data validation and parsing using Python type hints**.

It converts untrusted input data (JSON, dicts, API payloads) into **validated Python objects**.

Typical flow:

JSON input
↓
Parsing
↓
Type validation
↓
Python object (BaseModel instance)

Example:

```
from pydantic import BaseModel

class User(BaseModel):
    name: str
    age: int

data = {"name": "Alice", "age": "25"}

user = User(**data)
```

Result:

age is automatically converted to int.

Q2. Why is Pydantic heavily used in FastAPI?

Answer:

FastAPI uses Pydantic models for:

• request validation
• response validation
• automatic OpenAPI schema generation
• serialization and deserialization

Example request body validation:

```
@app.post("/users")
def create_user(user: User):
    return user
```

FastAPI automatically validates request JSON using Pydantic.

==========
SECTION 2 — BASEMODEL
==========

Q3. What is BaseModel?

Answer:

BaseModel is the core class in Pydantic used to define data models.

It provides:

• type validation
• data parsing
• serialization
• error handling

Example:

```
from pydantic import BaseModel

class Product(BaseModel):
    id: int
    name: str
```

price: float

Q4. What happens when a BaseModel is instantiated?

Answer:

When creating a Pydantic model:

product = Product(id="1", name="Book", price="10.5")

Steps executed:

1. Parse input data
2. Convert types where possible
3. Validate field constraints
4. Raise ValidationError if invalid
5. Return Python object

==========
SECTION 3 — VALIDATION PIPELINE
==========

Q5. Explain the Pydantic validation pipeline.

Answer:

Pydantic validation flow:

Input data (JSON / dict)
↓
Field parsing
↓
Type coercion
↓
Field validators
↓
Model validators
↓
Final Python object

Example:

Input JSON

```
{
  "name": "Alice",
  "age": "30"
}
```

Pydantic converts:

age: "30" → 30

==========
SECTION 4 — FIELD CONFIGURATION
==========

Q6. What is Field() used for?

Answer:

Field() adds metadata and validation rules to model fields.

Example:

```
from pydantic import BaseModel, Field

class User(BaseModel):
    age: int = Field(gt=0, lt=120)
```

Constraints:

gt → greater than
lt → less than

Q7. Common Field constraints.

Answer:

Examples:

```
Field(gt=0)
Field(lt=100)
Field(min_length=3)
```

Field(max_length=50)
Field(regex="^[a-z]+$")

==========
SECTION 5 — CUSTOM VALIDATORS
==========

Q8. What are validators in Pydantic?

Answer:

Validators are functions that perform custom validation logic.

Example:

```
from pydantic import BaseModel, field_validator

class User(BaseModel):

    age: int

    @field_validator("age")
    def check_age(cls, value):
        if value < 18:
            raise ValueError("Must be adult")
        return value
```

Q9. Difference between field validators and model validators.

Answer:

Field validator

Validates individual fields.

Model validator

Validates relationships between fields.

Example:

```
class User(BaseModel):
```

```
    password: str
    confirm_password: str

    @model_validator(mode="after")
    def check_passwords(self):
        if self.password != self.confirm_password:
            raise ValueError("Passwords do not match")
        return self
```

==========
SECTION 6 — NESTED MODELS
==========

Q10. What are nested models?

Answer:

Pydantic models can contain other models as fields.

Example:

```
class Address(BaseModel):
    city: str
    country: str

class User(BaseModel):
    name: str
    address: Address
```

Input JSON:

```
{
 "name": "Alice",
 "address": {
   "city": "Paris",
   "country": "France"
 }
}
```

Q11. Why nested models are useful?

Answer:

They help represent complex hierarchical data structures.

Common in:

• API request bodies
• configuration systems
• database schemas

==========
SECTION 7 — SERIALIZATION
==========

Q12. What is model serialization?

Answer:

Serialization converts Pydantic models to dictionaries or JSON.

Example:

user.model_dump()

Returns dictionary.

user.model_dump_json()

Returns JSON string.

Q13. Why serialization is important?

Answer:

Needed for:

• sending API responses
• storing data
• logging

==========
SECTION 8 — ORM MODE
==========

Q14. What is ORM mode?

Answer:

ORM mode allows Pydantic models to read data directly from ORM objects.

Example:

```
class UserSchema(BaseModel):

    id: int
    name: str

    model_config = {
        "from_attributes": True
    }
```

This allows converting ORM objects to Pydantic models.

Q15. Example with SQLAlchemy

Answer:

```
user = db_user_object

schema = UserSchema.model_validate(user)
```

==========
SECTION 9 — ERROR HANDLING
==========

Q16. What happens when validation fails?

Answer:

Pydantic raises ValidationError.

Example:

User(age="abc")

Error:

ValidationError:
age
  Input should be a valid integer

==========
SECTION 10 — PERFORMANCE
==========

Q17. Why Pydantic is slower than plain dataclasses?

Answer:

Because Pydantic performs:

• runtime validation
• type conversion
• error generation

These checks add overhead.

Q18. Why Pydantic v2 became faster?

Answer:

Pydantic v2 uses a Rust-based validation engine called:

pydantic-core

Benefits:

• faster validation
• lower memory overhead
• improved parsing performance

==========
SECTION 11 — BEST PRACTICES
==========

Q19. Best practices when using Pydantic.

Answer:

• Use strict types when needed
• Avoid heavy validators in hot paths
• Separate input and output models
• Use nested models for complex schemas
• Validate external input always

Q20. When should you NOT use Pydantic?

Answer:

Avoid when:

• extremely performance-sensitive loops
• internal data structures without validation needs
• small scripts where validation overhead is unnecessary

PHASE — FASTAPI REQUEST AND RESPONSE LIFECYCLE
DEEP INTERVIEW Q&A (INTERNAL FLOW + PRACTICAL USAGE)
==========

SECTION 1 — OVERVIEW OF FASTAPI REQUEST LIFECYCLE

Q1. What happens internally when a request hits a FastAPI application?

Answer:

The request lifecycle in FastAPI follows these steps:

1. Client sends HTTP request
2. ASGI server (Uvicorn/Hypercorn) receives request
3. ASGI server passes request to FastAPI application
4. FastAPI routes request to correct endpoint

5. Request data is parsed and validated using Pydantic
6. Dependency injection is executed
7. Path operation function runs
8. Response model serialization occurs
9. Response object is generated
10. ASGI server sends response back to client

Q2. What role does ASGI play in FastAPI?

Answer:

FastAPI is built on the ASGI standard (Asynchronous Server Gateway Interface).

ASGI allows:

• asynchronous request handling
• concurrency with event loops
• high-performance I/O

ASGI servers include:

• Uvicorn
• Hypercorn
• Daphne

ASGI communicates using three components:

scope
receive
send

==========
SECTION 2 — REQUEST OBJECT
==========

Q3. What is the Request object in FastAPI?

Answer:

The Request object represents the incoming HTTP request.

It provides access to:

• headers
• cookies
• body
• query parameters
• client information

Example:

```
from fastapi import Request

@app.get("/")
async def read_root(request: Request):
    return {"client": request.client.host}
```

Q4. What information can be accessed from the Request object?

Answer:

Common attributes:

request.method
HTTP method (GET, POST, etc)

request.url
Full request URL

request.headers
HTTP headers

request.cookies
Cookie values

request.query_params
Query parameters

request.client
Client IP information

Q5. How do you read raw request body?

Answer:

Use request.body().

Example:

body = await request.body()

Returns raw bytes of request body.

==========
SECTION 3 — RESPONSE OBJECTS
==========

Q6. What is a Response object in FastAPI?

Answer:

Response objects define the outgoing HTTP response.

They contain:

• status code
• headers
• body
• cookies

Q7. What is JSONResponse?

Answer:

JSONResponse returns JSON formatted responses.

Example:

```
from fastapi.responses import JSONResponse

return JSONResponse(
    content={"message": "success"},
    status_code=200
)
```

Q8. What is PlainTextResponse?

Answer:

PlainTextResponse sends plain text instead of JSON.

Example:

```
from fastapi.responses import PlainTextResponse

return PlainTextResponse("Hello world")
```

Q9. What is StreamingResponse?

Answer:

StreamingResponse streams data incrementally instead of loading entire content in memory.

Useful for:

- large files
- real-time data streams
- video/audio streaming

Example:

```
from fastapi.responses import StreamingResponse

def generate():
    yield "chunk1"
    yield "chunk2"

return StreamingResponse(generate())
```

Q10. What is FileResponse?

Answer:

FileResponse sends files directly to the client.

Example:

```
from fastapi.responses import FileResponse

return FileResponse("report.pdf")
```

FastAPI automatically sets:

• Content-Type
• Content-Length
• streaming

==========
SECTION 4 — HEADERS
==========

Q11. How do you read request headers?

Answer:

Headers are available via request.headers.

Example:

```
@app.get("/")
async def get_header(request: Request):
    user_agent = request.headers.get("user-agent")
    return {"ua": user_agent}
```

Q12. How do you send custom headers in a response?

Answer:

Example:

```
from fastapi import Response
```

```
response = Response(content="ok")
response.headers["X-App-Version"] = "1.0"

return response
```

==========
SECTION 5 — COOKIES
==========

Q13. How do you set cookies in FastAPI?

Answer:

Example:

```
from fastapi import Response

@app.get("/set-cookie")
def set_cookie(response: Response):
    response.set_cookie(key="session", value="abc123")
    return {"message": "cookie set"}
```

Q14. How do you read cookies?

Answer:

Example:

```
@app.get("/read-cookie")
def read_cookie(request: Request):
    session = request.cookies.get("session")
    return {"session": session}
```

==========
SECTION 6 — BACKGROUND TASKS
==========

Q15. What are BackgroundTasks in FastAPI?

Answer:

BackgroundTasks allow tasks to run after sending a response.

Useful for:

• sending emails
• logging
• file processing

Q16. Example of BackgroundTasks

Answer:

```
from fastapi import BackgroundTasks

def send_email(email):
    print("sending email")

@app.post("/notify")
def notify(background_tasks: BackgroundTasks):
    background_tasks.add_task(send_email, "user@example.com")
    return {"message": "Email scheduled"}
```

Q17. Why use background tasks instead of running tasks directly?

Answer:

Benefits:

• response returned immediately
• long tasks do not block request
• improves API responsiveness

==========
SECTION 7 — RESPONSE FLOW SUMMARY
==========

Q18. Complete request-response lifecycle in FastAPI

Answer:

Client Request
↓
ASGI server receives request
↓
FastAPI router matches endpoint
↓
Dependencies executed
↓
Pydantic validation
↓
Endpoint function runs
↓
Response model serialization
↓
Response object created
↓
ASGI sends response to client

==========
END OF FASTAPI REQUEST / RESPONSE LIFECYCLE
==========
PHASE — FASTAPI DEPENDENCY INJECTION SYSTEM (THE CORE SUPERPOWER)
DEEP INTERVIEW Q&A (INTERNAL MECHANICS + PRACTICAL DESIGN)
==========

SECTION 1 — WHY DEPENDENCY INJECTION EXISTS

Q1. What is Dependency Injection (DI) in FastAPI?

Answer:

Dependency Injection is a design pattern where required objects or services are provided to a function automatically rather than created inside it.

FastAPI implements DI using the Depends() function.

Example:

```
from fastapi import Depends

def get_db():
    return "db_connection"
```

```
@app.get("/users")
def get_users(db=Depends(get_db)):
    return {"db": db}
```

FastAPI automatically calls get_db() and injects the result into the endpoint.

Benefits:

• separation of concerns
• reusable components
• easier testing
• cleaner architecture

Q2. Why is FastAPI's DI system considered powerful?

Answer:

FastAPI's DI system is powerful because it:

• automatically builds a dependency graph
• resolves dependencies recursively
• supports async and sync dependencies
• supports caching within a request
• supports cleanup using yield

This allows complex service graphs to be managed automatically.

==========
SECTION 2 — Depends()
==========

Q3. What does Depends() do?

Answer:

Depends() tells FastAPI that a parameter should be provided by another function.

Example:

```
def get_current_user():
    return {"name": "Alice"}
```

```
@app.get("/profile")
def profile(user=Depends(get_current_user)):
    return user
```

FastAPI resolves the dependency before executing the endpoint.

Q4. When are dependencies executed?

Answer:

Dependencies execute before the path operation function.

Execution order:

1. resolve dependencies
2. resolve sub-dependencies
3. run endpoint function

==========
SECTION 3 — DEPENDENCY GRAPH
==========

Q5. What is a dependency graph?

Answer:

FastAPI builds a dependency graph automatically.

Example:

Endpoint depends on Auth
Auth depends on DB
DB depends on Config

Graph:

Endpoint
↓
Auth dependency
↓
Database dependency

↓
Configuration dependency

FastAPI resolves this graph automatically.

Q6. What are sub-dependencies?

Answer:

Dependencies can depend on other dependencies.

Example:

```
def get_config():
    return {"env": "prod"}

def get_db(config=Depends(get_config)):
    return f"db_using_{config}"

def get_user(db=Depends(get_db)):
    return {"db": db}
```

FastAPI resolves the chain automatically.

==========
SECTION 4 — SCOPED DEPENDENCIES
==========

Q7. What are request-scoped dependencies?

Answer:

Request-scoped dependencies run once per request.

Example:

```
def get_db():
    return database_connection
```

FastAPI caches the dependency for the duration of the request.

Q8. What are application-scoped dependencies?

Answer:

Application-scoped dependencies are created once when the app starts.

Example:

database connection pools
configuration objects
machine learning models

These are typically initialized during application startup events.

==========
SECTION 5 — DEPENDENCY CACHING
==========

Q9. How does dependency caching work?

Answer:

FastAPI automatically caches dependency results during a request.

Example:

```
def get_db():
    print("creating db connection")
    return "db"

@app.get("/")
def endpoint(
    db1=Depends(get_db),
    db2=Depends(get_db)
):
    pass
```

Output:

creating db connection

get_db runs only once per request.

Q10. How can caching be disabled?

Answer:

By using:

Depends(get_db, use_cache=False)

This forces FastAPI to execute the dependency every time.

==========
SECTION 6 — YIELD DEPENDENCIES
==========

Q11. What are yield dependencies?

Answer:

Dependencies using yield behave like context managers.

Example:

```
def get_db():

    db = create_connection()

    try:
        yield db
    finally:
        db.close()
```

Flow:

Before request:
connection opened

After response:
cleanup code runs

Q12. Why are yield dependencies important?

Answer:

They allow safe resource management.

Examples:

• database connections
• transactions
• file handles
• background cleanup

==========
SECTION 7 — REAL WORLD EXAMPLES
==========

Q13. Database dependency example

Answer:

```
def get_db():

    db = SessionLocal()

    try:
        yield db
    finally:
        db.close()
```

Q14. Authentication dependency example

Answer:

```
def get_current_user(token=Depends(oauth2_scheme)):
    user = verify_token(token)
    return user

@app.get("/me")
def read_me(user=Depends(get_current_user)):
    return user
```

==========
SECTION 8 — ADVANCED DEPENDENCY PATTERNS
==========

Q15. Dependency injection for services

Example:

```
def get_user_service(db=Depends(get_db)):
    return UserService(db)
```

This allows separation of business logic from API layer.

Q16. Dependency overrides (testing)

Answer:

FastAPI allows overriding dependencies during testing.

Example:

```
app.dependency_overrides[get_db] = fake_db
```

==========
SECTION 9 — COMPLETE DEPENDENCY FLOW
==========

Q17. Full dependency resolution flow

Answer:

Request arrives
↓
FastAPI inspects endpoint parameters
↓
Detects dependencies via Depends()
↓
Builds dependency graph
↓

Resolves sub-dependencies
↓
Executes dependencies
↓
Caches results
↓
Runs endpoint
↓
Executes cleanup for yield dependencies

==========
END OF FASTAPI DEPENDENCY INJECTION PHASE
==========